arXiv:1911.05722v3 [cs.CV] 23 Mar 2020

Momentum Contrast for Unsupervised Visual Representation Learning

Kaiming He Haoqi Fan Yuxin Wu Saining Xie Ross Girshick

Facebook AI Research (FAIR)

Code: https://github.com/facebookresearch/moco

Abstract

We present Momentum Contrast (MoCo) for unsupervised visual representation learning. From a perspective on contrastive learning [29] as dictionary look-up, we build a dynamic dictionary with a queue and a moving-averaged encoder. This enables building a large and consistent dictionary on-the-fly that facilitates contrastive unsupervised learning. MoCo provides competitive results under the common linear protocol on ImageNet classification. More importantly, the representations learned by MoCo transfer well to downstream tasks. MoCo can ***outperform*** *its supervised pre-training counterpart in 7 detection/segmentation tasks on PASCAL VOC, COCO, and other datasets, sometimes surpassing it by large margins. This suggests that the gap between unsupervised and supervised representation learning has been largely closed in many vision tasks.*

1. Introduction

Unsupervised representation learning is highly successful in natural language processing, *e.g.*, as shown by GPT [50, 51] and BERT [12]. But supervised pre-training is still dominant in computer vision, where unsupervised methods generally lag behind. The reason may stem from differences in their respective signal spaces. Language tasks have discrete signal spaces (words, sub-word units, *etc.*) for building tokenized *dictionaries*, on which unsupervised learning can be based. Computer vision, in contrast, further concerns dictionary building [54, 9, 5], as the raw signal is in a continuous, high-dimensional space and is not structured for human communication (*e.g.*, unlike words).

Several recent studies [61, 46, 36, 66, 35, 56, 2] present promising results on unsupervised visual representation learning using approaches related to the *contrastive loss* [29]. Though driven by various motivations, these methods can be thought of as building *dynamic dictionaries*. The "keys" (tokens) in the dictionary are sampled from data (*e.g.*, images or patches) and are represented by an encoder network. Unsupervised learning trains encoders to perform dictionary look-up: an encoded "query" should be similar to its matching key and dissimilar to others. Learning is formulated as minimizing a contrastive loss [29].

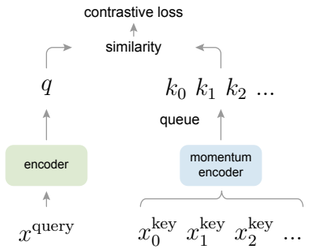


Figure 1. Momentum Contrast (MoCo) trains a visual representation encoder by matching an encoded query q to a dictionary of encoded keys using a contrastive loss. The dictionary keys $\{k_0, k_1, k_2, ...\}$ are defined on-the-fly by a set of data samples. The dictionary is built as a queue, with the current mini-batch enqueued and the oldest mini-batch dequeued, decoupling it from the mini-batch size. The keys are encoded by a slowly progressing encoder, driven by a momentum update with the query encoder. This method enables a large and consistent dictionary for learning visual representations.

From this perspective, we hypothesize that it is desirable to build dictionaries that are: (i) large *and* (ii) consistent as they evolve during training. Intuitively, a larger dictionary may better sample the underlying continuous, high-dimensional visual space, while the keys in the dictionary should be represented by the same or similar encoder so that their comparisons to the query are consistent. However, existing methods that use contrastive losses can be limited in one of these two aspects (discussed later in context).

We present *Momentum Contrast* (MoCo) as a way of building large and consistent dictionaries for unsupervised learning with a contrastive loss (Figure 1). We maintain the dictionary as a *queue* of data samples: the encoded representations of the current mini-batch are enqueued, and the oldest are dequeued. The queue decouples the dictionary size from the mini-batch size, allowing it to be large. Moreover, as the dictionary keys come from the preceding several mini-batches, a *slowly progressing* key encoder, implemented as a momentum-based moving average of the query encoder, is proposed to maintain consistency.

arXiv:1911.05722v3 [cs.CV] 23 Mar 2020

用于无监督视觉表征学习的动量对比

KaimingHe HaoqiFan Yuxin Wu Saining Xie Ross Girshick

Facebook AI Research (FAIR)

代码：https://github.com/facebookresearch/moco

摘要

我们提出用于无监督视觉表征学习的动量对比（*MoCo*）。将对比学习 *[29]*看作字典查找，通过队列与移动平均编码器构建动态字典，从而在线建立规模大、编码一致的字典，促进无监督对比学习。*MoCo* 在常用 *ImageNet* 线性评估中具有竞争力。更重要的是，其表征能良好迁移到下游任务：在 *PASCAL VOC*、*COCO* 等数据集的7项检测或分割任务中，*MoCo* 可超过对应监督预训练，有时优势明显。这说明，在多种视觉任务中，无监督与监督表征学习的差距已大幅缩小。

1 引言

无监督表征学习在自然语言处理中非常成功，例如 GPT[50, 51] 和 BERT [12]。但计算机视觉仍以监督预训练为主，无监督方法通常落后。原因可能来自信号空间差异。语言的离散信号空间，如词和子词，可直接构建词元化字典，无监督学习以此为基础。视觉则还需要解决字典构建 [54, 9, 5], 问题：原始信号位于连续、高维空间，也不像词语那样，天然具有服务于人类交流的结构。

近期多项研究 [61, 46, 36, 66, 35, 56, 2] 通过与对比损失有关的方法[29]，在无监督视觉表征学习上取得有希望的结果。尽管动机不同，这些方法都可看作构建动态字典。字典的“键”，即词元，从图像或图像块等数据中抽取，由编码器表示。无监督学习训练编码器进行字典查找：编码后的查询应接近匹配的键，远离其他键。学习可表述为最小化对比损失 [。

29].

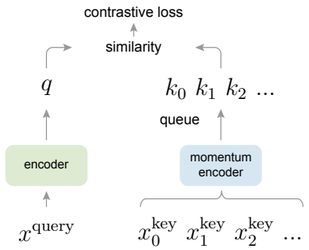


图1：动量对比 MoCo 通过对比损失，将编码查询 q 与编码键字典匹配，训练视觉编码器。字典中的键 $\{k_0, k_1, k_2, ...\}$ 由样本集合在线定义。字典维护为队列，当前批次入队，最旧批次出队，因此大小与批量分离。键由缓慢变化的编码器编码，该编码器通过查询编码器的动量更新驱动。由此得到规模大、编码一致的字典，用于学习视觉表征。

从这一角度出发，我们假设理想的字典应当：（i）规模大；（ii）在随训练演化时保持一致。直观上，更大的字典可以更充分地采样底层连续、高维的视觉空间；同时，字典中的键应由相同或相近的编码器表示，使其与查询之间的比较保持一致。然而，现有使用对比损失的方法可能在这两个方面之一受到限制（后文将结合具体背景讨论）。

我们提出动量对比 MoCo，通过对比损失构建大且一致的字典，用于无监督学习（图1）。将字典维护为样本队列：当前批次的编码表征入队，最旧的表征出队。队列使字典大小与批量大小分离，因此字典可以很大。由于键来自此前多个批次，我们提出缓慢变化的键编码器，以查询编码器的动量移动平均实现，从而保持编码一致性。

MoCo is a mechanism for building dynamic dictionaries for contrastive learning, and can be used with various pretext tasks. In this paper, we follow a simple instance discrimination task [61, 63, 2]: a query matches a key if they are encoded views (*e.g.*, different crops) of the same image. Using this pretext task, MoCo shows competitive results under the common protocol of linear classification in the ImageNet dataset [11].

A main purpose of unsupervised learning is to pre-train representations (*i.e.*, features) that can be transferred to downstream tasks by fine-tuning. We show that in 7 downstream tasks related to detection or segmentation, MoCo unsupervised pre-training can *surpass* its ImageNet supervised counterpart, in some cases by nontrivial margins. In these experiments, we explore MoCo pre-trained on ImageNet or on a *one-billion* Instagram image set, demonstrating that MoCo can work well in a more real-world, billion-image scale, and relatively uncurated scenario. These results show that MoCo largely closes the gap between unsupervised and supervised representation learning in many computer vision tasks, and can serve as an alternative to ImageNet supervised pre-training in several applications.

2. Related Work

Unsupervised/self-supervised[1] learning methods generally involve two aspects: pretext tasks and loss functions. The term "pretext" implies that the task being solved is not of genuine interest, but is solved only for the true purpose of learning a good data representation. Loss functions can often be investigated independently of pretext tasks. MoCo focuses on the loss function aspect. Next we discuss related studies with respect to these two aspects.

Loss functions. A common way of defining a loss function is to measure the difference between a model's prediction and a *fixed* target, such as reconstructing the input pixels (*e.g.*, auto-encoders) by L1 or L2 losses, or classifying the input into pre-defined categories (*e.g.*, eight positions [13], color bins [64]) by cross-entropy or margin-based losses. Other alternatives, as described next, are also possible.

Contrastive losses [29] measure the similarities of sample pairs in a representation space. Instead of matching an input to a fixed target, in contrastive loss formulations the target can *vary* on-the-fly during training and can be defined in terms of the data representation computed by a network [29]. Contrastive learning is at the core of several recent works on unsupervised learning [61, 46, 36, 66, 35, 56, 2], which we elaborate on later in context (Sec. 3.1).

Adversarial losses [24] measure the difference between probability distributions. It is a widely successful technique for unsupervised data generation. Adversarial methods for representation learning are explored in [15, 16]. There are relations (see [24]) between generative adversarial networks and noise-contrastive estimation (NCE) [28].

Pretext tasks. A wide range of pretext tasks have been proposed. Examples include recovering the input under some corruption, *e.g.*, denoising auto-encoders [58], context auto-encoders [48], or cross-channel auto-encoders (colorization) [64, 65]. Some pretext tasks form pseudo-labels by, *e.g.*, transformations of a single ("exemplar") image [17], patch orderings [13, 45], tracking [59] or segmenting objects [47] in videos, or clustering features [3, 4].

Contrastive learning *vs.* pretext tasks. Various pretext tasks can be based on some form of contrastive loss functions. The instance discrimination method [61] is related to the exemplar-based task [17] and NCE [28]. The pretext task in contrastive predictive coding (CPC) [46] is a form of context auto-encoding [48], and in contrastive multiview coding (CMC) [56] it is related to colorization [64].

3. Method

3.1. Contrastive Learning as Dictionary Look-up

Contrastive learning [29], and its recent developments, can be thought of as training an encoder for a *dictionary look-up* task, as described next.

Consider an encoded query q and a set of encoded samples $\{k_0, k_1, k_2, ...\}$ that are the keys of a dictionary. Assume that there is a single key (denoted as k_+) in the dictionary that q matches. A contrastive loss [29] is a function whose value is low when q is similar to its positive key k_+ and dissimilar to all other keys (considered negative keys for q). With similarity measured by dot product, a form of a contrastive loss function, called InfoNCE [46], is considered in this paper:

$$\mathcal{L}_q = -\log \frac{\exp(q \cdot k_+ / \tau)}{\sum_{i=0}^{K} \exp(q \cdot k_i / \tau)} \tag{1}$$

where τ is a temperature hyper-parameter per [61]. The sum is over one positive and K negative samples. Intuitively, this loss is the log loss of a $(K+1)$-way softmax-based classifier that tries to classify q as k_+. Contrastive loss functions can also be based on other forms [29, 59, 61, 36], such as margin-based losses and variants of NCE losses.

The contrastive loss serves as an unsupervised objective function for training the encoder networks that represent the queries and keys [29]. In general, the query representation is $q = f_{\mathrm{q}}(x^q)$ where f_{q} is an encoder network and x^q is a query sample (likewise, $k = f_{\mathrm{k}}(x^k)$). Their instantiations depend on the specific pretext task. The input x^q and x^k can be images [29, 61, 63], patches [46], or context consisting a set of patches [46]. The networks f_{q} and f_{k} can be identical [29, 59, 63], partially shared [46, 36, 2], or different [56].

[1] Self-supervised learning is a form of unsupervised learning. Their distinction is informal in the existing literature. In this paper, we use the more classical term of "unsupervised learning", in the sense of "not supervised by *human*-annotated labels".

MoCo 是为对比学习构建动态字典的机制，可以配合多种代理任务。本文采用简单的实例判别任务 [61, 63, 2]：如果查询与键是同一图像的不同编码视图（如不同裁剪），则二者匹配。利用这一代理任务，MoCo 在 ImageNet 数据集常用的线性分类协议下取得了有竞争力的结果 [11]。

无监督学习的一个主要目的，是预训练可经微调迁移到下游任务的表征，即特征。在7项检测或分割下游任务中，MoCo 无监督预训练可超过对应 ImageNet 监督预训练，有些差距明显。实验分别在 ImageNet 和10亿张 Instagram 图像上预训练，表明 MoCo 能用于更接近现实、十亿规模且未经精细筛选的数据。这些结果显示，MoCo 大幅缩小了多项视觉任务上无监督与监督表征学习的差距，并可在若干应用中替代 ImageNet 监督预训练。

2 相关工作

无监督／自监督[1] 学习方法通常涉及两个方面：代理任务和损失函数。“代理”意味着所解决的任务本身并非真正的研究目标，解决它只是为了学习良好的数据表征。损失函数通常可独立于代理任务研究。MoCo 关注损失函数这一方面。下面从这两个角度讨论相关研究。

损失函数。一种常见的定义方式是衡量模型预测与固定目标之间的差异，例如用 L1 或 L2 损失重建输入像素（如自编码器），或用交叉熵或基于间隔的损失，将输入归入预定义类别（如八个位置 [13]，、颜色分箱 [64]）。也可以采用下文所述的其他方式。

对比损失 [29] 衡量表征空间中样本对的相似度。对比损失并不将输入匹配到固定目标，而是允许目标在训练过程中动态变化，且目标可以由网络计算出的数据表征来定义[29]。对比学习是近期多项无监督学习研究 [61, 46, 36, 66, 35, 56, 2],的核心；后文将结合具体背景进一步讨论（第3.1节）。

对抗损失 [24]衡量概率分布之间的差异，是一种广泛取得成功的技术，可用于来进行无监督数据生成。文献 [15, 16]探索了用于表征学习的对抗方法。生成对抗网络与噪声对比估计（NCE） [28]之间存在联系（见 [24]）。

代理任务。已有研究提出了各种代理任务。例如，在输入受到某种破坏后恢复输入，包括去噪自编码器 [58], 、上下文自编码器 [48], 或跨通道自编码器（着色） [64, 65]。一些代理任务通过以下方式构造伪标签：变换单张（“样本”）图像 [17],、排列图像块 [13, 45], 、在视频中跟踪 [59]或分割物体 [47], 或对特征进行聚类 [3, 4]。

对比学习与代理任务。多种代理任务可以采用某种形式的对比损失。实例判别方法 [61]与基于样本的任务 [17] 及 NCE [28]有关。对比预测编码（CPC） [46] 中的代理任务是一种上下文自编码 [48], ；对比多视图编码（CMC） [56]的任务则与着色 [64]有关。

3 方法

3.1 将对比学习视为字典查询

对比学习 [29], 及其近期发展，可理解为训练一个编码器来完成字典查询任务，具体如下。

考虑编码查询 q，以及构成字典键的编码样本集合 $\{k_0, k_1, k_2, ...\}$。假设字典中只有一个键，记为 k_+，与 q 匹配。对比损失 [29]在 q 接近其正键 k_+、远离其他键，即 q 的负键时，取值较低。以点积衡量相似性，本文采用名为 InfoNCE [46], 的对比损失形式：

$$\mathcal{L}_q = -\log \frac{\exp(q \cdot k_+ / \tau)}{\sum_{i=0}^{K} \exp(q \cdot k_i / \tau)} \tag{1}$$

其中，τ是 [61]中的温度超参数。求和涵盖一个正样本和 K 个负样本。直观地说，该损失是一个基于 softmax 的（K+1）分类器的对数损失，它尝试将 q 分类为 k_+。对比损失还可以采用其他形式 [29, 59, 61, 36], , 例如基于间隔的损失以及 NCE 损失的变体。

对比损失作为无监督目标，训练表示查询与键的编码器 [29]。通常，查询表征为 $q = f_{\mathrm{q}}(x^q)$，其中 f_{q} 是编码器， x^q 是查询样本；同样有 $k = f_{\mathrm{k}}(x^k)$。具体实现由代理任务决定。输入 x^q 和 x^k 可以是图像 [29, 61, 63], 、图像块 [46], , 或由若干块组成的上下文 [46]。网络 f_{q} 与 f_{k} 可以相同[29, 59, 63], 、部分共享 [46, 36, 2],或完全不同 [56]。

[1] 自监督学习是无监督学习的一种。已有文献并未严格区分二者。本文使用更传统的“无监督学习”一词，指“不使用人工标注标签进行监督”。

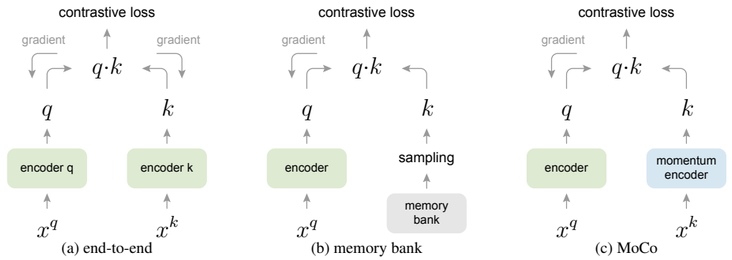


Figure 2. **Conceptual comparison of three contrastive loss mechanisms** (empirical comparisons are in Figure 3 and Table 3). Here we illustrate one pair of query and key. The three mechanisms differ in how the keys are maintained and how the key encoder is updated. **(a)**: The encoders for computing the query and key representations are updated ***end-to-end*** by back-propagation (the two encoders can be different). **(b)**: The key representations are sampled from a ***memory bank*** [61]. **(c)**: ***MoCo*** encodes the new keys on-the-fly by a momentum-updated encoder, and maintains a queue (not illustrated in this figure) of keys.

3.2. Momentum Contrast

From the above perspective, contrastive learning is a way of building a discrete dictionary on high-dimensional continuous inputs such as images. The dictionary is *dynamic* in the sense that the keys are randomly sampled, and that the key encoder evolves during training. Our hypothesis is that good features can be learned by a *large* dictionary that covers a rich set of negative samples, while the encoder for the dictionary keys is kept as *consistent* as possible despite its evolution. Based on this motivation, we present Momentum Contrast as described next.

Dictionary as a queue. At the core of our approach is maintaining the dictionary as a *queue* of data samples. This allows us to reuse the encoded keys from the immediate preceding mini-batches. The introduction of a queue decouples the dictionary size from the mini-batch size. Our dictionary size can be much larger than a typical mini-batch size, and can be flexibly and independently set as a hyper-parameter.

The samples in the dictionary are progressively replaced. The current mini-batch is enqueued to the dictionary, and the oldest mini-batch in the queue is removed. The dictionary always represents a sampled subset of all data, while the extra computation of maintaining this dictionary is manageable. Moreover, removing the oldest mini-batch can be beneficial, because its encoded keys are the most outdated and thus the least consistent with the newest ones.

Momentum update. Using a queue can make the dictionary large, but it also makes it intractable to update the key encoder by back-propagation (the gradient should propagate to all samples in the queue). A naïve solution is to copy the key encoder f_k from the query encoder f_q, ignoring this gradient. But this solution yields poor results in experiments (Sec. 4.1). We hypothesize that such failure is caused by the rapidly changing encoder that reduces the key representations' consistency. We propose a momentum update to address this issue.

Formally, denoting the parameters of f_k as θ_k and those of f_q as θ_q, we update θ_k by:

$$\theta_\mathrm{k} \leftarrow m\theta_\mathrm{k} + (1-m)\theta_\mathrm{q}. \tag{2}$$

Here $m \in [0, 1)$ is a momentum coefficient. Only the parameters θ_q are updated by back-propagation. The momentum update in Eqn.(2) makes θ_k evolve more smoothly than θ_q. As a result, though the keys in the queue are encoded by different encoders (in different mini-batches), the difference among these encoders can be made small. In experiments, a relatively large momentum (*e.g.*, $m = 0.999$, our default) works much better than a smaller value (*e.g.*, $m = 0.9$), suggesting that a slowly evolving key encoder is a core to making use of a queue.

Relations to previous mechanisms. MoCo is a general mechanism for using contrastive losses. We compare it with two existing general mechanisms in Figure 2. They exhibit different properties on the dictionary size and consistency.

The ***end-to-end*** update by back-propagation is a natural mechanism (*e.g.*, [29, 46, 36, 63, 2, 35], Figure 2a). It uses samples in the current mini-batch as the dictionary, so the keys are consistently encoded (by the same set of encoder parameters). But the dictionary size is coupled with the mini-batch size, limited by the GPU memory size. It is also challenged by large mini-batch optimization [25]. Some recent methods [46, 36, 2] are based on pretext tasks driven by local positions, where the dictionary size can be made larger by multiple positions. But these pretext tasks may require special network designs such as patchifying the input [46] or customizing the receptive field size [2], which may complicate the transfer of these networks to downstream tasks.

Another mechanism is the ***memory bank*** approach proposed by [61] (Figure 2b). A memory bank consists of the representations of all samples in the dataset. The dictionary for each mini-batch is randomly sampled from the memory bank with no back-propagation, so it can support a large dictionary size. However, the representation of a sample in

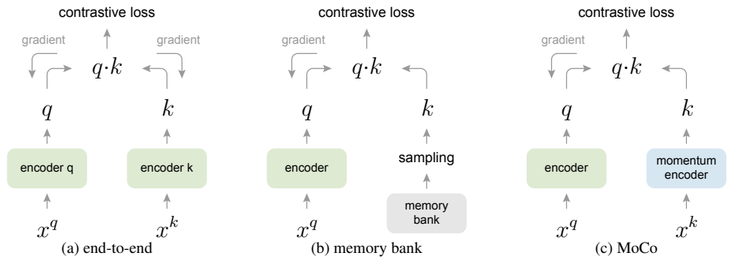


图2：三种对比损失机制的概念比较，实证比较见图3和表3。这里只展示一对查询和键。三种机制在键的维护和键编码器更新方式上不同。（a）计算查询与键的两个编码器通过反向传播端到端更新，可以采用不同编码器；（b）从记忆库 [61]抽取键表征；（c）MoCo 用动量更新的编码器在线编码新键，并维护键队列，图中未画出。

3.2 动量对比

从上述视角看，对比学习是在图像等高维连续输入上构建离散字典的一种方式。字典是动态的：键经过随机采样，键编码器也随训练演化。我们的假设是，较大的字典能覆盖丰富的负样本，从而学习良好特征；与此同时，尽管编码器不断演化，字典键的编码器仍应尽可能保持一致。基于这一动机，我们提出下文的动量对比方法。

将字典组织为队列。我们方法的核心是以数据样本队列维护字典，从而复用刚刚经过的若干小批量的编码键。队列使字典大小与小批量大小解耦。字典可以远大于常见的小批量，其大小可作为超参数灵活、独立地设置。

字典中的样本逐步替换：将当前小批量入队，并移除队列中最早的小批量。字典始终表示全部数据的一个采样子集，维护字典所需的额外计算量也可控。此外，移除最早的小批量可能有益，因为这些编码键最陈旧，与最新键的一致性最低。

动量更新。队列可以扩大字典，但也使通过反向传播更新键编码器变得不可行（梯度需要传播到队列中的所有样本）。一种朴素的方案是直接将查询编码器 f_q 复制给键编码器 f_k，忽略这部分梯度。但该方案的实验结果较差（第4.1节）。我们推测，这是因为编码器变化过快，降低了键表征的一致性。为解决这一问题，我们提出动量更新。

形式上，将 f_k 的参数记为 θ_k，将 f_q 的参数记为 θ_q，则按如下方式更新 θ_k：

$$\theta_\mathrm{k} \leftarrow m\theta_\mathrm{k} + (1-m)\theta_\mathrm{q}. \tag{2}$$

这里 $m \in [0, 1)$ 为动量系数。仅参数 θ_q 通过反向传播更新。式2的动量更新，使 θ_k 比 θ_q 变化更平滑。因此，尽管队列中的键由不同批次的不同编码器编码，这些编码器之间的差异仍可保持较小。实验中，较大动量，例如默认的 $m = 0.999$，明显优于较小值，如 $m = 0.9$。这表明，缓慢变化的键编码器，是有效使用队列的核心。

与已有机制的关系。MoCo 是使用对比损失的通用机制。图2将其与两种已有通用机制比较。它们在字典大小和一致性方面具有不同性质。

通过反向传播端到端更新是自然的机制，例如 [29, 46, 36, 63, 2, 35], , 见图2a。它以当前批次样本为字典，全部键由同一组参数编码，因此编码一致。但字典大小与批量绑定，受到 GPU 内存限制，也面临大批量优化挑战 [25]。近期一些方法 [46, 36, 2] 使用局部位置驱动的代理任务，通过多个位置扩大字典。但这些任务可能要求特殊设计，例如将输入分块 [46]或定制感受野 [2], , 使网络迁移到下游任务更复杂。

另一种机制是 [61]提出的记忆库方法（图2b）。记忆库保存数据集中所有样本的表征。每个小批量的字典从记忆库随机采样，且不进行反向传播，因此支持大字典。然而，某个样本在

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
   x_q = aug(x) # a randomly augmented version
   x_k = aug(x) # another randomly augmented version

   q = f_q.forward(x_q) # queries: NxC
   k = f_k.forward(x_k) # keys: NxC
   k = k.detach() # no gradient to keys

   # positive logits: Nx1
   l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

   # negative logits: NxK
   l_neg = mm(q.view(N,C), queue.view(C,K))

   # logits: Nx(1+K)
   logits = cat([l_pos, l_neg], dim=1)

   # contrastive loss, Eqn.(1)
   labels = zeros(N) # positives are the 0-th
   loss = CrossEntropyLoss(logits/t, labels)

   # SGD update: query network
   loss.backward()
   update(f_q.params)

   # momentum update: key network
   f_k.params = m*f_k.params+(1-m)*f_q.params

   # update dictionary
   enqueue(queue, k) # enqueue the current minibatch
   dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

the memory bank was updated when it was last seen, so the sampled keys are essentially about the encoders at multiple different steps all over the past epoch and thus are less consistent. A momentum update is adopted on the memory bank in [61]. Its momentum update is on the representations of the same sample, *not* the encoder. This momentum update is irrelevant to our method, because MoCo does not keep track of every sample. Moreover, our method is more memory-efficient and can be trained on billion-scale data, which can be intractable for a memory bank.

Sec. 4 empirically compares these three mechanisms.

3.3. Pretext Task

Contrastive learning can drive a variety of pretext tasks. As the focus of this paper is not on designing a new pretext task, we use a simple one mainly following the *instance discrimination* task in [61], to which some recent works [63, 2] are related.

Following [61], we consider a query and a key as a positive pair if they originate from the same image, and otherwise as a negative sample pair. Following [63, 2], we take two random "views" of the same image under random data augmentation to form a positive pair. The queries and keys are respectively encoded by their encoders, f_q and f_k. The encoder can be any convolutional neural network [39].

Algorithm 1 provides the pseudo-code of MoCo for this pretext task. For the current mini-batch, we encode the queries and their corresponding keys, which form the positive sample pairs. The negative samples are from the queue.

Technical details. We adopt a ResNet [33] as the encoder, whose last fully-connected layer (after global average pooling) has a fixed-dimensional output (128-D [61]). This output vector is normalized by its L2-norm [61]. This is the representation of the query or key. The temperature τ in Eqn.(1) is set as 0.07 [61]. The data augmentation setting follows [61]: a 224×224-pixel crop is taken from a randomly resized image, and then undergoes random color jittering, random horizontal flip, and random grayscale conversion, all available in PyTorch's torchvision package.

Shuffling BN. Our encoders f_q and f_k both have Batch Normalization (BN) [37] as in the standard ResNet [33]. In experiments, we found that using BN prevents the model from learning good representations, as similarly reported in [35] (which avoids using BN). The model appears to "cheat" the pretext task and easily finds a low-loss solution. This is possibly because the intra-batch communication among samples (caused by BN) leaks information.

We resolve this problem by shuffling BN. We train with multiple GPUs and perform BN on the samples independently for each GPU (as done in common practice). For the key encoder f_k, we shuffle the sample order in the current mini-batch before distributing it among GPUs (and shuffle back after encoding); the sample order of the mini-batch for the query encoder f_q is not altered. This ensures the batch statistics used to compute a query and its positive key come from two different subsets. This effectively tackles the cheating issue and allows training to benefit from BN.

We use shuffled BN in both our method and its end-to-end ablation counterpart (Figure 2a). It is irrelevant to the memory bank counterpart (Figure 2b), which does not suffer from this issue because the positive keys are from different mini-batches in the past.

4. Experiments

We study unsupervised training performed in:

***ImageNet-1M* (IN-1M)**: This is the ImageNet [11] training set that has ~1.28 million images in 1000 classes (often called ImageNet-1K; we count the image number instead, as classes are not exploited by unsupervised learning). This dataset is well-balanced in its class distribution, and its images generally contain iconic view of objects.

***Instagram-1B* (IG-1B)**: Following [44], this is a dataset of *~1 billion* (940M) public images from *Instagram*. The images are from ~1500 hashtags [44] that are related to the ImageNet categories. This dataset is relatively *uncurated* comparing to IN-1M, and has a *long-tailed*, *unbalanced* distribution of real-world data. This dataset contains both iconic objects and scene-level images.

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
   x_q = aug(x) # a randomly augmented version
   x_k = aug(x) # another randomly augmented version

   q = f_q.forward(x_q) # queries: NxC
   k = f_k.forward(x_k) # keys: NxC
   k = k.detach() # no gradient to keys

   # positive logits: Nx1
   l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

   # negative logits: NxK
   l_neg = mm(q.view(N,C), queue.view(C,K))

   # logits: Nx(1+K)
   logits = cat([l_pos, l_neg], dim=1)

   # contrastive loss, Eqn.(1)
   labels = zeros(N) # positives are the 0-th
   loss = CrossEntropyLoss(logits/t, labels)

   # SGD update: query network
   loss.backward()
   update(f_q.params)

   # momentum update: key network
   f_k.params = m*f_k.params+(1-m)*f_q.params

   # update dictionary
   enqueue(queue, k) # enqueue the current minibatch
   dequeue(queue) # dequeue the earliest minibatch
```

bmm：批量矩阵乘法；mm：矩阵乘法；cat：拼接。

记忆库中的条目是在对应样本上次出现时更新的，因此，采样得到的键实际上来自过去一个训练轮次中多个不同时刻的编码器，因而一致性较差。文献 [61]对记忆库采用了动量更新，但更新的是同一样本的表征，而非编码器。这种动量更新与我们的方法无关，因为 MoCo 不逐一跟踪每个样本。此外，我们的方法更节省内存，可以在十亿规模的数据上训练，而记忆库方法可能难以做到。

第4节通过实验比较这三种机制。

3.3 代理任务

对比学习可用于多种代理任务。本文不以设计新代理任务为重点，因此采用简单的任务，主要遵循 [61], 中的实例判别任务；近期一些研究 [63,2] 也与之相关。

按照 [61], , 若查询和键来自同一图像，则将它们视为正样本对，否则视为负样本对。按照 [63, 2], , 我们对同一图像进行随机数据增强，取两个随机"视图"组成正样本对。查询和键分别由各自的编码器 f_q 和 f_k 编码。编码器可以是任意卷积神经网络 [39]。

算法1给出了 MoCo 在该任务上的伪代码。

代理任务。对当前小批量，我们编码查询及其对应键，组成正样本对；负样本则来自队列。

技术细节。采用 ResNet [33] 编码器，全局平均池化后的最后全连接层输出固定的128维向量 [61]，并用 L2 范数归一化 [61]，作为查询或键的表征。式1的温度 τ 设为0.07 [61]。增强沿用 [61]：先随机缩放，再取 224×224 像素裁剪，然后随机颜色抖动、水平翻转及灰度转换，均由 PyTorch 的 torchvision 提供。

打乱 BN。我们的编码器 f_q 和 f_k 均使用批归一化（BN） [37]，与标准 ResNet [33]相同。实验发现，使用 BN 会妨碍模型学习良好的表征；文献 [35] 也报告了类似现象（并避免使用 BN）。模型似乎会在代理任务中"作弊"，轻易找到低损失解。这可能是因为 BN 导致批次内样本之间发生信息交流，从而泄露了信息。

我们通过打乱 BN 的样本顺序解决这一问题。训练使用多块 GPU，并按通常做法，在每块 GPU 内独立计算样本的 BN。对于键编码器 f_k，先打乱当前小批量中样本的顺序，再分发到各 GPU（编码后再恢复顺序）；查询编码器 f_q 的小批量顺序保持不变。这保证了计算查询及其正键所使用的批次统计量来自不同的两个子集。该方法有效解决了"作弊"问题，并使训练能够受益于 BN。

我们的方法及其端到端消融变体（图2a）均使用打乱的 BN。记忆库变体（图2b）不需要这一处理：其正键来自过去不同的小批量，因此不会遇到上述问题。

4 实验

我们研究在以下数据集上进行无监督训练：

ImageNet-1M（IN-1M）：这是 ImageNet [11]训练集，包含1000类、~1.28百万张图像（通常称为 ImageNet-1K；我们按图像数量命名，因为无监督学习不使用类别）。该数据集的类别分布均衡，图像通常呈现物体的典型视图。

Instagram-1B（IG-1B）：按照 [44], , 该数据集包含 Instagram 上的 *~1* 十亿张（9.4亿张）公开图像。图像来自与 ImageNet 类别相关的 ~1500 个话题标签 [44] 。与 IN-1M 相比，该数据集经过较少人工整理，其真实世界数据呈现长尾、不均衡的分布，既包含突出单个物体的图像，也包含场景级图像。

Training. We use SGD as our optimizer. The SGD weight decay is 0.0001 and the SGD momentum is 0.9. For IN-1M, we use a mini-batch size of 256 (N in Algorithm 1) in 8 GPUs, and an initial learning rate of 0.03. We train for 200 epochs with the learning rate multiplied by 0.1 at 120 and 160 epochs [61], taking ~53 hours training ResNet-50. For IG-1B, we use a mini-batch size of 1024 in 64 GPUs, and a learning rate of 0.12 which is exponentially decayed by 0.9× after every 62.5k iterations (64M images). We train for 1.25M iterations (~1.4 epochs of IG-1B), taking ~6 days for ResNet-50.

4.1. Linear Classification Protocol

We first verify our method by *linear* classification on *frozen* features, following a common protocol. In this subsection we perform unsupervised pre-training on IN-1M. Then we freeze the features and train a supervised linear classifier (a fully-connected layer followed by softmax). We train this classifier on the global average pooling features of a ResNet, for 100 epochs. We report 1-crop, top-1 classification accuracy on the ImageNet validation set.

For this classifier, we perform a grid search and find the optimal initial learning rate is 30 and weight decay is 0 (similarly reported in [56]). These hyper-parameters perform consistently well for all ablation entries presented in this subsection. These hyper-parameter values imply that the feature distributions (*e.g.*, magnitudes) can be substantially different from those of ImageNet supervised training, an issue we will revisit in Sec. 4.2.

Ablation: contrastive loss mechanisms. We compare the three mechanisms that are illustrated in Figure 2. To focus on the effect of contrastive loss mechanisms, we implement all of them in the same pretext task as described in Sec. 3.3. We also use the same form of InfoNCE as the contrastive loss function, Eqn.(1). As such, the comparison is solely on the three mechanisms.

The results are in Figure 3. Overall, all three mechanisms benefit from a larger K. A similar trend has been observed in [61, 56] under the memory bank mechanism, while here we show that this trend is more general and can be seen in all mechanisms. These results support our motivation of building a large dictionary.

The ***end-to-end*** mechanism performs similarly to MoCo when K is small. However, the dictionary size is limited by the mini-batch size due to the end-to-end requirement. Here the largest mini-batch a high-end machine (8 Volta 32GB GPUs) can afford is 1024. More essentially, large mini-batch training is an open problem [25]: we found it necessary to use the linear learning rate scaling rule [25] here, without which the accuracy drops (by ~2% with a 1024 mini-batch). But optimizing with a larger mini-batch is harder [25], and it is questionable whether the trend can be extrapolated into a larger K even if memory is sufficient.

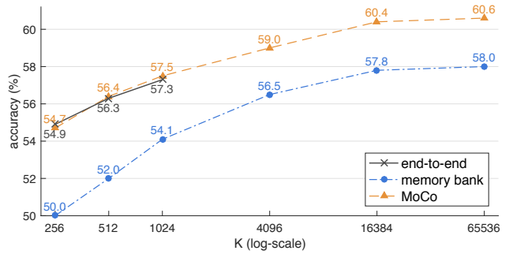


Figure 3. **Comparison of three contrastive loss mechanisms** under the ImageNet linear classification protocol. We adopt the same pretext task (Sec. 3.3) and only vary the contrastive loss mechanism (Figure 2). The number of negatives is K in memory bank and MoCo, and is $K-1$ in end-to-end (offset by one because the positive key is in the same mini-batch). The network is ResNet-50.

The ***memory bank*** [61] mechanism can support a larger dictionary size. But it is 2.6% worse than MoCo. This is inline with our hypothesis: the keys in the memory bank are from very different encoders all over the past epoch and they are not consistent. Note the memory bank result of 58.0% reflects our improved implementation of [61].[2]

Ablation: momentum. The table below shows ResNet-50 accuracy with different MoCo momentum values (m in Eqn.(2)) used in pre-training ($K = 4096$ here) :

momentum m	0	0.9	0.99	0.999	0.9999
accuracy (%)	*fail*	55.2	57.8	59.0	58.9

It performs reasonably well when m is in 0.99 ~ 0.9999, showing that a slowly progressing (*i.e.*, relatively large momentum) key encoder is beneficial. When m is too small (*e.g.*, 0.9), the accuracy drops considerably; at the extreme of *no momentum* (m is 0), the training loss oscillates and fails to converge. These results support our motivation of building a consistent dictionary.

Comparison with previous results. Previous unsupervised learning methods can differ substantially in model sizes. For a fair and comprehensive comparison, we report ***accuracy vs. #parameters***[3] trade-offs. Besides ResNet-50 (R50) [33], we also report its variants that are 2× and 4× wider (more channels), following [38].[4] We set $K = 65536$ and $m = 0.999$. Table 1 is the comparison.

MoCo with R50 performs competitively and achieves 60.6% accuracy, better than all competitors of similar model sizes (~24M). MoCo benefits from larger models and achieves 68.6% accuracy with R50w4×.

Notably, we achieve competitive results using a *standard* ResNet-50 and require no specific architecture designs, *e.g.*,

[2]Here 58.0% is with InfoNCE and K=65536. We reproduce 54.3% when using NCE and K=4096 (the same as [61]), close to 54.0% in [61].

[3]Parameters are of the *feature extractor*: *e.g.*, we do not count the parameters of $conv_x$ if $conv_x$ is not included in linear classification.

[4]Our w2× and w4× models correspond to the "×8" and "×16" cases in [38], because the standard-sized ResNet is referred to as "×4" in [38].

训练。优化器为 SGD，权重衰减0.0001，动量0.9。IN-1M 使用8块 GPU，批量256，即算法1中的 N，初始学习率0.03。训练200轮，在第120、160轮将学习率乘0.1 [61], ; ResNet-50 用时 ~53 小时。IG-1B 使用64块 GPU、批量1024、学习率0.12，每62500次迭代，即6400万图像，将学习率乘0.9× ，进行指数衰减。总计125万次迭代，约 IG-1B 的 ~1.4轮，ResNet-50 用时 ~6 天。

4.1 线性分类协议

我们首先遵循常用协议，在冻结特征上进行线性分类来验证方法。本小节在 IN-1M 上进行无监督预训练，然后冻结特征，训练有监督线性分类器（全连接层后接 softmax）。分类器以 ResNet 的全局平均池化特征为输入，训练100轮。我们报告 ImageNet 验证集上的单次裁剪 top-1 分类准确率。

我们对该分类器进行网格搜索，发现最佳初始学习率为30、权重衰减为0（文献 [56]也有类似报告）。这些超参数在本小节的所有消融实验中都表现稳定。它们表明，特征分布（如幅值）可能与 ImageNet 有监督训练得到的特征显著不同；第4.2节将再次讨论这一问题。

消融：对比损失机制。我们比较图2中的三种机制。为专注于机制本身的影响，三者均使用第3.3节所述的同一代理任务，也使用式（1）所示的相同 InfoNCE 对比损失。因此，比较仅涉及三种机制的差异。

结果见图3。总体上，三种机制都受益于更大的 K。文献 [61, 56] 在记忆库机制下观察到了类似趋势；这里我们表明，这一趋势更为普遍，三种机制中均可见。这些结果支持了我们构建大字典的动机。

当 K 较小时，端到端机制与 MoCo 表现相近。但端到端要求使字典大小受限于小批量大小。此处一台高端机器（8块32GB Volta GPU）所能支持的最大小批量为1024。更根本地说，大批量训练仍是开放问题 [25]：我们发现此处必须使用学习率线性缩放规则 [25]，否则准确率会下降（小批量为1024时下降 ~2个百分点）。然而，较大小批量的优化更困难 [25], ; 即便内存足够，这一趋势能否外推到更大的 K 也值得怀疑。

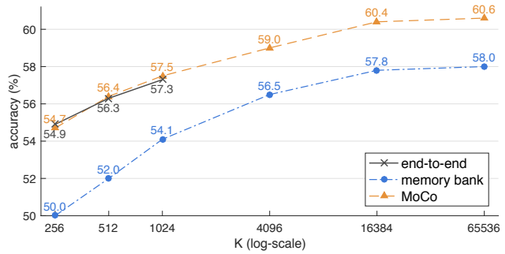


图3：在 ImageNet 线性分类协议下比较三种对比损失机制。我们使用相同的代理任务（第3.3节），只改变对比损失机制（图2）。记忆库和 MoCo 的负样本数为 K，端到端方法为 $K-1$（减去一个，因为正键位于同一小批量内）。网络为 ResNet-50。

记忆库 [61] 机制支持较大的字典，但比 MoCo 低2.6个百分点。这符合我们的假设：记忆库中的键来自过去整个训练轮次中差异较大的编码器，缺乏一致性。注意，58.0%的记忆库结果来自我们对 [61]的改进实现。[2]

消融：动量。下表展示预训练中使用不同 MoCo 动量值（式（2）中的m ）时的 ResNet-50 准确率（此处 $K = 4096$ ）：

动量 m	0	0.9	0.99	0.999	0.9999
准确率（%）	*fail*	55.2	57.8	59.0	58.9

当 m 处于0.99 ~ 0.9999范围时，表现较好，说明缓慢演化（即动量较大）的键编码器是有益的。当 m 过小（如0.9）时，准确率明显下降；在完全没有动量的极端情况（m 为0）下，训练损失振荡，无法收敛。这些结果支持了我们构建一致字典的动机。

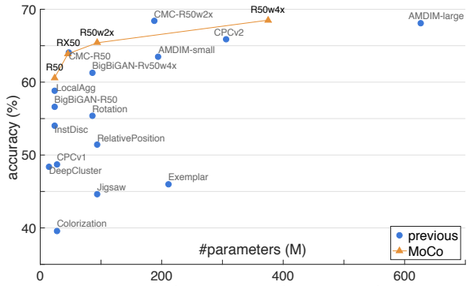
与已有结果比较。已有无监督学习方法的模型规模可能差异很大。为公平、全面地比较，我们报告准确率与参数量[3] 之间的权衡。除了 ResNet-50（R50） [33], , 我们还按照 [38]报告通道数扩大 2× 和 4×倍的变体。[4] 我们设置 $K = 65536$及 $m = 0.999$。比较见表1。

采用 R50 的 MoCo 达到有竞争力的60.6%准确率，优于所有模型规模相近（~24百万参数）的对比方法。MoCo 受益于更大的模型，采用 R50w4×时达到68.6%的准确率。

值得注意的是，我们使用标准 ResNet-50 就能获得有竞争力的结果，无需特定架构设计，例如

[2]这里58.0%采用 InfoNCE 与 K=65536。使用 NCE 与 K=4096 ，即和 [61]相同设置，我们复现54.3%，接近 [61]的54.0%。

[3]参数量只计特征提取器，例如线性分类不使用 $conv_x$ 时，不计 $conv_x$ 。[4]我们的 w2× 、w4× 对应 [38],中的"×8"和"×16"，因为 [38]将标准大小 ResNet 称为"×4" 。



method	architecture	#params (M)	accuracy (%)
Exemplar [17]	R50w3×	211	46.0 [38]
RelativePosition [13]	R50w2×	94	51.4 [38]
Jigsaw [45]	R50w2×	94	44.6 [38]
Rotation [19]	Rv50w4×	86	55.4 [38]
Colorization [64]	R101*	28	39.6 [14]
DeepCluster [3]	VGG [53]	15	48.4 [4]
BigBiGAN [16]	R50	24	56.6
	Rv50w4×	86	61.3
methods based on contrastive learning follow:			
InstDisc [61]	R50	24	54.0
LocalAgg [66]	R50	24	58.8
CPC v1 [46]	R101*	28	48.7
CPC v2 [35]	$R170^*_{wider}$	303	65.9
CMC [56]	$R50_{L+ab}$	47	64.1†
	$R50w2\times_{L+ab}$	188	68.4†
AMDIM [2]	$AMDIM_{small}$	194	63.5†
	$AMDIM_{large}$	626	68.1†
MoCo	R50	24	60.6
	RX50	46	63.9
	R50w2×	94	65.4
	R50w4×	375	**68.6**

Table 1. **Comparison under the linear classification protocol on ImageNet**. The figure visualizes the table. All are reported as unsupervised pre-training on the ImageNet-1M training set, followed by supervised linear classification trained on frozen features, evaluated on the validation set. The parameter counts are those of the feature extractors. We compare with improved re-implementations if available (referenced after the numbers).
Notations: R101*/R170* is ResNet-101/170 with the last residual stage removed [14, 46, 35], and R170 is made wider [35]; Rv50 is a reversible net [23], RX50 is ResNeXt-50-32×8d [62].
†: *Pre-training uses FastAutoAugment [40] that is supervised by ImageNet labels.*

patchified inputs [46, 35], carefully tailored receptive fields [2], or combining two networks [56]. By using an architecture that is not customized for the pretext task, it is easier to transfer features to a variety of visual tasks and make comparisons, studied in the next subsection.

This paper's focus is on a mechanism for general contrastive learning; we do not explore orthogonal factors (such as specific pretext tasks) that may further improve accuracy. As an example, "MoCo v2" [8], an extension of a preliminary version of this manuscript, achieves 71.1% accuracy with R50 (up from 60.6%), given small changes on the data augmentation and output projection head [7]. We believe that this additional result shows the generality and robustness of the MoCo framework.

pre-train	AP_{50}	AP	AP_{75}
random init.	64.4	37.9	38.6
super. IN-1M	81.4	54.0	59.1
MoCo IN-1M	81.1 (−0.3)	54.6 (+**0.6**)	59.9 (+**0.8**)
MoCo IG-1B	81.6 (+0.2)	55.5 (+**1.5**)	61.2 (+**2.1**)

(a) Faster R-CNN, R50-**dilated-C5**

pre-train	AP_{50}	AP	AP_{75}
random init.	60.2	33.8	33.1
super. IN-1M	81.3	53.5	58.8
MoCo IN-1M	81.5 (+0.2)	55.9 (+**2.4**)	62.6 (+**3.8**)
MoCo IG-1B	82.2 (+**0.9**)	57.2 (+**3.7**)	63.7 (+**4.9**)

(b) Faster R-CNN, R50-**C4**

Table 2. **Object detection fine-tuned on PASCAL VOC** `trainval07+12`. Evaluation is on `test2007`: AP_{50} (default VOC metric), AP (COCO-style), and AP_{75}, averaged over 5 trials. All are fine-tuned for 24k iterations (~23 epochs). In the brackets are the gaps to the ImageNet supervised pre-training counterpart. In green are the gaps of at least +**0.5** point.

	R50-dilated-C5			R50-C4		
pre-train	AP_{50}	AP	AP_{75}	AP_{50}	AP	AP_{75}
end-to-end	79.2	52.0	56.6	80.4	54.6	60.3
memory bank	79.8	52.9	57.9	80.6	54.9	60.6
MoCo	**81.1**	**54.6**	**59.9**	**81.5**	**55.9**	**62.6**

Table 3. **Comparison of three contrastive loss mechanisms** on PASCAL VOC object detection, fine-tuned on `trainval07+12` and evaluated on `test2007` (averages over 5 trials). All models are implemented by us (Figure 3), pre-trained on IN-1M, and fine-tuned using the same settings as in Table 2.

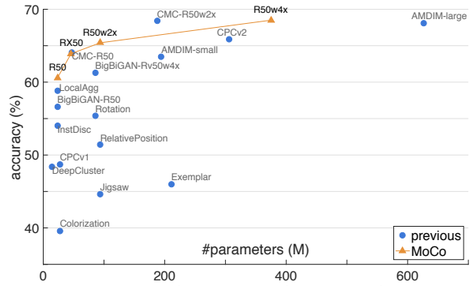
4.2. Transferring Features

A main goal of unsupervised learning is to *learn features that are transferrable*. ImageNet supervised pre-training is most influential when serving as the initialization for fine-tuning in downstream tasks (*e.g.*, [21, 20, 43, 52]). Next we compare MoCo with ImageNet supervised pre-training, transferred to various tasks including PASCAL VOC [18], COCO [42], *etc*. As prerequisites, we discuss two important issues involved [31]: normalization and schedules.

Normalization. As noted in Sec. 4.1, features produced by unsupervised pre-training can have different distributions compared with ImageNet supervised pre-training. But a system for a downstream task often has hyper-parameters (*e.g.*, learning rates) selected for supervised pre-training. To relieve this problem, we adopt *feature normalization* during fine-tuning: we fine-tune with BN that is trained (and synchronized across GPUs [49]), instead of freezing it by an affine layer [33]. We also use BN in the newly initialized layers (*e.g.*, FPN [41]), which helps calibrate magnitudes.

We perform normalization when fine-tuning supervised and unsupervised pre-training models. MoCo *uses the same hyper-parameters* as the ImageNet supervised counterpart.

Schedules. If the fine-tuning schedule is long enough, training detectors from random initialization can be strong baselines, and can match the ImageNet supervised counterpart on COCO [31]. Our goal is to investigate *transferabil-*



method	architecture	#params (M)	accuracy (%)
Exemplar [17]	R50w3×	211	46.0 [38]
RelativePosition [13]	R50w2×	94	51.4 [38]
Jigsaw [45]	R50w2×	94	44.6 [38]
Rotation [19]	Rv50w4×	86	55.4 [38]
Colorization [64]	R101*	28	39.6 [14]
DeepCluster [3]	VGG [53]	15	48.4 [4]
BigBiGAN [16]	R50	24	56.6
	Rv50w4×	86	61.3
methods based on contrastive learning follow:			
InstDisc [61]	R50	24	54.0
LocalAgg [66]	R50	24	58.8
CPC v1 [46]	R101*	28	48.7
CPC v2 [35]	$R170^*_{wider}$	303	65.9
CMC [56]	$R50_{L+ab}$	47	64.1†
	$R50w2\times_{L+ab}$	188	68.4†
AMDIM [2]	$AMDIM_{small}$	194	63.5†
	$AMDIM_{large}$	626	68.1†
MoCo	R50	24	60.6
	RX50	46	63.9
	R50w2×	94	65.4
	R50w4×	375	**68.6**

可视化。所有结果均先在 ImageNet-1M 训练集上无监督预训练，再冻结特征，训练有监督线性分类器，并在验证集上评估。参数量为特征提取器的参数量。如有改进后的重新实现结果，我们使用这些结果进行比较（引用标注在数值后）。

记号：R101*／R170* 表示 ResNet 101/170，并去除最后一个残差阶段 [14, 46, 35]，；R170 还增加了网络宽度 [35]；Rv50 是可逆网络 [23]，；RX50 是 ResNeXt-50-32×8d [62]。

†：预训练使用 *FastAutoAugment [40]*，其中使用了 *ImageNet* 标签监督。

分块输入 [46, 35]，、精心设计的感受野[2]，，或结合两个网络 [56]。采用不针对代理任务定制的架构，更容易将特征迁移到多种视觉任务并进行比较；下一小节将对此研究。

本文关注通用对比学习机制，没有探索可能进一步提高精度的其他独立因素（如具体代理任务）。例如，"MoCo v2" [8]，是在本文早期版本基础上的扩展，只对数据增强和输出投影头做了少量修改 [7]，便使 R50 的准确率由60.6%提高到71.1%。我们认为，这一补充结果体现了 MoCo 框架的通用性和稳健性。

预训练	AP_{50}	AP	AP_{75}
随机初始化	64.4	37.9	38.6
有监督 IN-1M	81.4	54.0	59.1
MoCo IN-1M	81.1 (−0.3)	54.6 (+**0.6**)	59.9 (+**0.8**)
MoCo IG-1B	81.6 (+0.2)	55.5 (+**1.5**)	61.2 (+**2.1**)

（a）Faster R-CNN，R50-dilated-C5

预训练	AP_{50}	AP	AP_{75}
随机初始化	60.2	33.8	33.1
有监督 IN-1M	81.3	53.5	58.8
MoCo IN-1M	81.5 (+0.2)	55.9 (+**2.4**)	62.6 (+**3.8**)
MoCo IG-1B	82.2 (+**0.9**)	57.2 (+**3.7**)	63.7 (+**4.9**)

（b）Faster R-CNN，R50-C4

表2：在 PASCAL VOC trainval`07+12`上微调的目标检测结果。评估使用 test2007，报告 AP_{50}（VOC 默认指标）、AP（COCO 风格）和 AP_{75}，，取5次试验的平均值。所有模型均微调24千次迭代（~23 轮）。括号中为相对于对应 ImageNet 有监督预训练模型的差值；至少达到 +**0**.5个百分点的增益用绿色表示。

	R50-dilated-C5			R50-C4		
预训练	AP_{50}	AP	AP_{75}	AP_{50}	AP	AP_{75}
端到端	79.2	52.0	56.6	80.4	54.6	60.3
记忆库	79.8	52.9	57.9	80.6	54.9	60.6
MoCo	**81.1**	**54.6**	**59.9**	**81.5**	**55.9**	**62.6**

表3：在 PASCAL VOC 目标检测上比较三种对比损失机制。在 trainval`07+12` 上微调，在 test2007 上评估，取5次试验平均值。全部模型均由我们实现（图3），在 IN-1M 上预训练，使用与表2相同的微调设置。

4.2 特征迁移

无监督学习的主要目标之一是学习可迁移的特征。ImageNet 有监督预训练最重要的用途，是为下游任务微调提供初始化（例如 [21, 20, 43, 52]）。接下来，我们将 MoCo 与 ImageNet 有监督预训练进行比较，将二者迁移到 PASCAL VOC [18]，、COCO [42], 等多种任务。首先讨论其中两个重要问题 [31]：归一化和训练日程。

归一化。如第4.1节所述，无监督预训练产生的特征分布可能与 ImageNet 有监督预训练不同。但下游任务系统的超参数（如学习率）往往是针对有监督预训练选择的。为缓解这一问题，我们在微调时采用特征归一化：让 BN 参与训练（并在 GPU 之间同步 [49]），而不是用仿射层将其固定 [33]。我们还在新初始化的层（如 FPN [41]）中使用 BN，这有助于校准特征的幅值。

在微调有监督和无监督预训练模型时，我们都使用归一化。MoCo 采用与对应 ImageNet 有监督预训练模型相同的超参数。

训练日程。如果微调日程足够长，从随机初始化训练的检测器也能成为强基线，并在 COCO 上追平 ImageNet 有监督预训练模型 [31]。我们的目标是研究特征的可迁移

pre-train	AP_{50} RelPos, by [14]	Multi-task [14]	Jigsaw, by [26]	LocalAgg [66]	**MoCo**	AP **MoCo**	AP_{75} Multi-task [14]	**MoCo**
super. IN-1M	74.2	74.2	70.5	74.6	74.4	42.4	44.3	42.7
unsup. IN-1M	66.8 (−7.4)	70.5 (−3.7)	61.4 (−9.1)	69.1 (−5.5)	74.9 (+**0.5**)	46.6 (+**4.2**)	43.9 (−0.4)	50.1 (+**7.4**)
unsup. IN-14M	-	-	69.2 (−1.3)	-	75.2 (+**0.8**)	46.9 (+**4.5**)	-	50.2 (+**7.5**)
unsup. YFCC-100M	-	-	66.6 (−3.9)	-	74.7 (+0.3)	45.9 (+**3.5**)	-	49.0 (+**6.3**)
unsup. IG-1B	-	-	-	-	75.6 (+**1.2**)	47.6 (+**5.2**)	-	51.7 (+**9.0**)

Table 4. **Comparison with previous methods on object detection fine-tuned on PASCAL VOC** `trainval2007`. Evaluation is on `test2007`. The ImageNet supervised counterparts are from the respective papers, and are reported as having *the same structure* as the respective unsupervised pre-training counterparts. All entries are based on the C4 backbone. The models in [14] are R101 v2 [34], and others are R50. The RelPos (relative position) [13] result is the best single-task case in the Multi-task paper [14]. The Jigsaw [45] result is from the ResNet-based implementation in [26]. Our results are with 9k-iteration fine-tuning, averaged over 5 trials. In the brackets are the gaps to the ImageNet supervised pre-training counterpart. In green are the gaps of at least +**0.5** point.

ity of features, so our experiments are on controlled schedules, *e.g.*, the 1× (~12 epochs) or 2× schedules [22] for COCO, in contrast to 6×~9× in [31]. On smaller datasets like VOC, training longer may not catch up [31].

Nonetheless, in our fine-tuning, MoCo *uses the same schedule* as the ImageNet supervised counterpart, and random initialization results are provided as references.

Put together, our fine-tuning uses the same setting as the supervised pre-training counterpart. This may place MoCo at a *disadvantage*. Even so, MoCo is competitive. Doing so also makes it feasible to present comparisons on multiple datasets/tasks, without extra hyper-parameter search.

4.2.1 PASCAL VOC Object Detection

Setup. The detector is Faster R-CNN [52] with a backbone of R50-dilated-C5 or R50-C4 [32] (details in appendix), with BN tuned, implemented in [60]. We fine-tune all layers end-to-end. The image scale is [480, 800] pixels during training and 800 at inference. The same setup is used for all entries, including the supervised pre-training baseline. We evaluate the default VOC metric of AP_{50} (*i.e.*, IoU threshold is 50%) and the more stringent metrics of COCO-style AP and AP_{75}. Evaluation is on the VOC `test2007` set.

Ablation: backbones. Table 2 shows the results fine-tuned on `trainval07+12` (~16.5k images). For R50-dilated-C5 (Table 2a), MoCo pre-trained on IN-1M is comparable to the supervised pre-training counterpart, and MoCo pre-trained on IG-1B *surpasses* it. For R50-C4 (Table 2b), MoCo with IN-1M or IG-1B is *better* than the supervised counterpart: up to **+0.9** AP_{50}, **+3.7** AP, and **+4.9** AP_{75}.

Interestingly, the transferring accuracy depends on the detector structure. For the C4 backbone, by default used in existing ResNet-based results [14, 61, 26, 66], the advantage of unsupervised pre-training is larger. The relation between pre-training *vs*. detector structures has been veiled in the past, and should be a factor under consideration.

Ablation: contrastive loss mechanisms. We point out that these results are partially because we establish solid detection baselines for contrastive learning. To pin-point the gain that is *solely* contributed by using the MoCo mechanism in contrastive learning, we fine-tune the models pre-trained with the end-to-end or memory bank mechanism, both implemented by us (*i.e.*, the best ones in Figure 3), using the same fine-tuning setting as MoCo.

These competitors perform decently (Table 3). Their AP and AP_{75} with the C4 backbone are also higher than the ImageNet supervised counterpart's, *c.f*. Table 2b, but other metrics are lower. They are worse than MoCo in all metrics. This shows the benefits of MoCo. In addition, how to train these competitors in larger-scale data is an open question, and they may not benefit from IG-1B.

Comparison with previous results. Following the competitors, we fine-tune on `trainval2007` (~5k images) using the C4 backbone. The comparison is in Table 4.

For the AP_{50} metric, *no* previous method can catch up with its respective supervised pre-training counterpart. MoCo pre-trained on any of IN-1M, IN-14M (full ImageNet), YFCC-100M [55], and IG-1B can *outperform* the supervised baseline. Large gains are seen in the more stringent metrics: up to **+5.2** AP and **+9.0** AP_{75}. These gains are larger than the gains seen in `trainval07+12` (Table 2b).

4.2.2 COCO Object Detection and Segmentation

Setup. The model is Mask R-CNN [32] with the FPN [41] or C4 backbone, with BN tuned, implemented in [60]. The image scale is in [640, 800] pixels during training and is 800 at inference. We fine-tune all layers end-to-end. We fine-tune on the `train2017` set (~118k images) and evaluate on `val2017`. The schedule is the default 1× or 2× in [22].

Results. Table 5 shows the results on COCO with the FPN (Table 5a, b) and C4 (Table 5c, d) backbones. With the 1× schedule, all models (including the ImageNet supervised counterparts) are heavily under-trained, as indicated by the ~2 points gaps to the 2× schedule cases. With the 2× schedule, MoCo is *better* than its ImageNet supervised counterpart in all metrics in both backbones.

4.2.3 More Downstream Tasks

Table 6 shows more downstream tasks (implementation details in appendix). Overall, MoCo performs competitively

预训练	AP_{50} RelPos, by [14]	多任务 [14]	拼图，来自 [26]	LocalAgg [66]	**MoCo**	AP **MoCo**	AP_{75} 多任务 [14]	**MoCo**
有监督 IN-1M	74.2	74.2	70.5	74.6	74.4	42.4	44.3	42.7
无监督 IN-1M	66.8 (−7.4)	70.5 (−3.7)	61.4 (−9.1)	69.1 (−5.5)	74.9 (+**0.5**)	46.6 (+**4.2**)	43.9 (−0.4)	50.1 (+**7.4**)
无监督 IN-14M	-	-	69.2 (−1.3)	-	75.2 (+**0.8**)	46.9 (+**4.5**)	-	50.2 (+**7.5**)
无监督 YFCC-100M	-	-	66.6 (−3.9)	-	74.7 (+0.3)	45.9 (+**3.5**)	-	49.0 (+**6.3**)
无监督 IG-1B	-	-	-	-	75.6 (+**1.2**)	47.6 (+**5.2**)	-	51.7 (+**9.0**)

表4：在 PASCAL VOC trainval2007 微调的目标检测，与此前方法比较；在 test2007 评估。监督 ImageNet 对应结果来自各自论文，其结构与无监督预训练版本相同。全部使用 C4 主干。 [14] 中的模型为 R101 v2 [34],，其余为 R50。RelPos，相对位置 [13]，取 Multi-task 论文 [14]中最佳单任务结果。Jigsaw [45] 来自 [26]的 ResNet 实现。我们的结果为微调9000次、5次运行平均。括号中是相对监督 ImageNet 预训练的差距，绿色表示至少 +**0**.5点的提升。

性，因此实验采用受控的训练日程，如 COCO 上的 1× （~12 轮）或 2× 训练日程 [22]，而不是 [31]中的 6×~9× 日程。在 VOC 等较小数据集上，延长训练也可能无法追平 [31]。

不过，在我们的微调中，MoCo 使用与对应 ImageNet 有监督预训练模型相同的训练日程，并提供随机初始化结果作为参考。

总之，我们的微调使用与对应有监督预训练模型相同的设置。这可能对 MoCo 不利，但即便如此，MoCo 仍有竞争力。这也使我们无需额外超参数搜索，就能在多个数据集和任务上比较。

4.2.1 PASCAL VOC 目标检测

设置。检测器为 Faster R-CNN [52]，主干为 R50-dilated-C5 或 R50-C4 [32]，细节见附录；BN 可更新，使用 [60]实现。全部层端到端微调。训练图像尺度为 [480, 800] 像素，推理为800。所有条目包括监督基线采用相同设置。我们评估 VOC 默认 AP_{50}，即 IoU 阈值50%，以及更严格的 COCO 风格 AP、AP_{75}。评估集为 VOC test2007。

主干消融。表2为在 trainval`07+12` 、~16.5千张图像上微调的结果。R50-dilated-C5（表2a）中，IN-1M 上的 MoCo 与监督预训练相当，IG-1B 上则超过它。
R50-C4（表2b）中，无论 IN-1M 还是 IG-1B，MoCo 都更好，最高提升 +**0**.9 AP50、 +**3**.7 AP 和 +**4**.9 AP_{75}。

有趣的是，迁移准确率取决于检测器结构。对于现有基于 ResNet 的结果 [14, 61, 26, 66]，默认使用的 C4 骨干，无监督预训练的优势更大。预训练方式与检测器结构之间的关系过去未充分显现，应作为一个需要考虑的因素。

消融：对比损失机制。我们指出，这些结果部分得益于为对比学习建立了可靠的检测基线。为确定仅由 MoCo 机制贡献的

在对比学习中的增益，我们用自己实现的端到端或记忆库机制预训练模型（即图3中各自最佳的模型），采用与 MoCo 相同的微调设置进行微调。

这些对比方法表现尚可（表3）。采用 C4 骨干时，它们的 AP 和 AP_{75} 也超过了 ImageNet 有监督预训练结果（参见表2b），但其他指标较低。它们在所有指标上都逊于 MoCo，体现了 MoCo 的优势。此外，如何在更大规模数据上训练这些方法仍是一个开放问题，它们未必能从 IG-1B 中受益。

与已有结果比较。遵循对比方法，我们使用 C4 骨干，在 trainval2007（~5千张图像）上微调。比较见表4。

对于 AP_{50} 指标，此前没有方法能达到各自对应的有监督预训练结果。而在 IN-1M、IN-14M（完整 ImageNet）、YFCC-100M [55], 或 IG-1B 上预训练的 MoCo 均可超过有监督基线。在更严格的指标上，增益尤其明显：AP 最高提高 +**5**.2，AP_{75}最高提高 +**9**.0。这些增益大于在 trainval`07+12` 上观察到的增益（表2b）。

4.2.2 COCO 目标检测与分割

设置。模型为 Mask R-CNN [32]，骨干采用 FPN [41]或 C4，BN 参与微调，使用 [60]实现。训练时图像尺度为 [640, 800] 像素，推理时为800像素。所有层都进行端到端微调。我们在 train2017 集（~118千张图像）上微调，在 val2017 上评估。训练日程采用 [22]中的默认 1× 或 2× 设置。

结果。表5给出了采用 FPN（表5a、b）和 C4（表5c、d）骨干的 COCO 结果。采用1× 训练日程时，所有模型（包括 ImageNet 有监督预训练模型）都明显训练不足：与 2× 训练日程相比，存在 ~2个百分点的差距。采用2× 训练日程时，在两种骨干网络的所有指标上，MoCo 均优于对应的 ImageNet 有监督预训练模型。

4.2.3 更多下游任务

表6展示了更多下游任务（实现细节见附录）。总体上，MoCo 表现具有竞争力。

pre-train	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
random init.	31.0	49.5	33.2	28.5	46.8	30.4
super. IN-1M	38.9	59.6	42.7	35.4	56.5	38.1
MoCo IN-1M	38.5 (−0.4)	58.9 (−0.7)	42.0 (−0.7)	35.1 (−0.3)	55.9 (−0.6)	37.7 (−0.4)
MoCo IG-1B	38.9 (0.0)	59.4 (−0.2)	42.3 (−0.4)	35.4 (0.0)	56.5 (0.0)	37.9 (−0.2)

(a) Mask R-CNN, R50-**FPN**, **1**× schedule

pre-train	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
random init.	36.7	56.7	40.0	33.7	53.8	35.9
super. IN-1M	40.6	61.3	44.4	36.8	58.1	39.5
MoCo IN-1M	40.8 (+0.2)	61.6 (+0.3)	44.7 (+0.3)	36.9 (+0.1)	58.4 (+0.3)	39.7 (+0.2)
MoCo IG-1B	41.1 (+**0.5**)	61.8 (+**0.5**)	45.1 (+**0.7**)	37.4 (+**0.6**)	59.1 (+**1.0**)	40.2 (+**0.7**)

(b) Mask R-CNN, R50-**FPN**, **2**× schedule

pre-train	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
random init.	26.4	44.0	27.8	29.3	46.9	30.8
super. IN-1M	38.2	58.2	41.2	33.3	54.7	35.2
MoCo IN-1M	38.5 (+0.3)	58.3 (+0.1)	41.6 (+0.4)	33.6 (+0.3)	54.8 (+0.1)	35.6 (+0.4)
MoCo IG-1B	39.1 (+**0.9**)	58.7 (+**0.5**)	42.2 (+**1.0**)	34.1 (+**0.8**)	55.4 (+**0.7**)	36.4 (+**1.2**)

(c) Mask R-CNN, R50-**C4**, **1**× schedule

pre-train	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
random init.	35.6	54.6	38.2	31.4	51.5	33.5
super. IN-1M	40.0	59.9	43.1	34.7	56.5	36.9
MoCo IN-1M	40.7 (+**0.7**)	60.5 (+**0.6**)	44.1 (+**1.0**)	35.4 (+**0.7**)	57.3 (+**0.8**)	37.6 (+**0.7**)
MoCo IG-1B	41.1 (+**1.1**)	60.7 (+**0.8**)	44.8 (+**1.7**)	35.6 (+**0.9**)	57.4 (+**0.9**)	38.1 (+**1.2**)

(d) Mask R-CNN, R50-**C4**, **2**× schedule

Table 5. **Object detection and instance segmentation fine-tuned on COCO**: bounding-box AP (AP^{bb}) and mask AP (AP^{mk}) evaluated on `val2017`. In the brackets are the gaps to the ImageNet supervised pre-training counterpart. In green are the gaps of at least +**0.5** point.

pre-train	**COCO keypoint detection** AP^{kp}	AP^{kp}_{50}	AP^{kp}_{75}
random init.	65.9	86.5	71.7
super. IN-1M	65.8	86.9	71.9
MoCo IN-1M	66.8 (+**1.0**)	87.4 (+**0.5**)	72.5 (+**0.6**)
MoCo IG-1B	66.9 (+**1.1**)	87.8 (+**0.9**)	73.0 (+**1.1**)

pre-train	**COCO dense pose estimation** AP^{dp}	AP^{dp}_{50}	AP^{dp}_{75}
random init.	39.4	78.5	35.1
super. IN-1M	48.3	85.6	50.6
MoCo IN-1M	50.1 (+**1.8**)	86.8 (+**1.2**)	53.9 (+**3.3**)
MoCo IG-1B	50.6 (+**2.3**)	87.0 (+**1.4**)	54.3 (+**3.7**)

pre-train	**LVIS v0.5 instance segmentation** AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
random init.	22.5	34.8	23.8
super. IN-1M†	24.4	37.8	25.8
MoCo IN-1M	24.1 (−0.3)	37.4 (−0.4)	25.5 (−0.3)
MoCo IG-1B	24.9 (+**0.5**)	38.2 (+0.4)	26.4 (+**0.6**)

pre-train	**Cityscapes instance seg.** AP^{mk}	AP^{mk}_{50}	**Semantic seg.** (mIoU) **Cityscapes**	**VOC**
random init.	25.4	51.1	65.3	39.5
super. IN-1M	32.9	59.6	74.6	74.4
MoCo IN-1M	32.3 (−0.6)	59.3 (−0.3)	75.3 (+**0.7**)	72.5 (−1.9)
MoCo IG-1B	32.9 (0.0)	60.3 (+**0.7**)	75.5 (+**0.9**)	73.6 (−0.8)

Table 6. **MoCo *vs*. ImageNet supervised pre-training, fine-tuned on various tasks**. For each task, the same architecture and schedule are used for all entries (see appendix). In the brackets are the gaps to the ImageNet supervised pre-training counterpart. In green are the gaps of at least +**0.5** point.
†: this entry is with BN frozen, which improves results; see main text.

with ImageNet supervised pre-training:

COCO keypoint detection: supervised pre-training has *no* clear advantage over random initialization, whereas MoCo outperforms in all metrics.

COCO dense pose estimation [1]: MoCo substantially outperforms supervised pre-training, *e.g*., by 3.7 points in AP^{dp}_{75}, in this highly localization-sensitive task.

LVIS v0.5 instance segmentation [27]: this task has ~1000 long-tailed distributed categories. Specifically in LVIS for the ImageNet supervised baseline, we find fine-tuning with frozen BN (24.4 AP^{mk}) is better than tunable BN (details in appendix). So we compare MoCo with the better supervised pre-training variant in this task. MoCo with IG-1B surpasses it in all metrics.

Cityscapes instance segmentation [10]: MoCo with IG-1B is on par with its supervised pre-training counterpart in AP^{mk}, and is higher in AP^{mk}_{50}.

Semantic segmentation: On Cityscapes [10], MoCo outperforms its supervised pre-training counterpart by up to 0.9 point. But on VOC semantic segmentation, MoCo is worse by at least 0.8 point, a negative case we have observed.

Summary. In sum, MoCo can ***outperform*** its ImageNet supervised pre-training counterpart in **7** detection or segmentation tasks.[5] Besides, MoCo is on par on Cityscapes instance segmentation, and lags behind on VOC semantic segmentation; we show another comparable case on iNaturalist [57] in appendix. Overall, *MoCo has largely closed the gap between unsupervised and supervised representation learning in multiple vision tasks*.

Remarkably, in all these tasks, MoCo pre-trained on IG-1B is consistently better than MoCo pre-trained on IN-1M. This shows that *MoCo can perform well on this large-scale, relatively uncurated dataset*. This represents a scenario towards *real-world* unsupervised learning.

5. Discussion and Conclusion

Our method has shown positive results of unsupervised learning in a variety of computer vision tasks and datasets. A few open questions are worth discussing. MoCo's improvement from IN-1M to IG-1B is consistently noticeable but relatively small, suggesting that the larger-scale data may not be fully exploited. We hope an advanced pretext task will improve this. Beyond the simple instance discrimination task [61], it is possible to adopt MoCo for pretext tasks like masked auto-encoding, *e.g*., in language [12] and in vision [46]. We hope MoCo will be useful with other pretext tasks that involve contrastive learning.

[5]Namely, object detection on VOC/COCO, instance segmentation on COCO/LVIS, keypoint detection on COCO, dense pose on COCO, and semantic segmentation on Cityscapes.

预训练	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
随机初始化	31.0	49.5	33.2	28.5	46.8	30.4
有监督 IN-1M	38.9	59.6	42.7	35.4	56.5	38.1
MoCo IN-1M	38.5 (−0.4)	58.9 (−0.7)	42.0 (−0.7)	35.1 (−0.3)	55.9 (−0.6)	37.7 (−0.4)
MoCo IG-1B	38.9 (0.0)	59.4 (−0.2)	42.3 (−0.4)	35.4 (0.0)	56.5 (0.0)	37.9 (−0.2)

（a）Mask R-CNN，R50-FPN，**1**×训练日程

预训练	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
随机初始化	36.7	56.7	40.0	33.7	53.8	35.9
有监督 IN-1M	40.6	61.3	44.4	36.8	58.1	39.5
MoCo IN-1M	40.8 (+0.2)	61.6 (+0.3)	44.7 (+0.3)	36.9 (+0.1)	58.4 (+0.3)	39.7 (+0.2)
MoCo IG-1B	41.1 (+**0.5**)	61.8 (+**0.5**)	45.1 (+**0.7**)	37.4 (+**0.6**)	59.1 (+**1.0**)	40.2 (+**0.7**)

（b）Mask R-CNN，R50-FPN，**2**× 训练日程

预训练	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
随机初始化	26.4	44.0	27.8	29.3	46.9	30.8
有监督 IN-1M	38.2	58.2	41.2	33.3	54.7	35.2
MoCo IN-1M	38.5 (+0.3)	58.3 (+0.1)	41.6 (+0.4)	33.6 (+0.3)	54.8 (+0.1)	35.6 (+0.4)
MoCo IG-1B	39.1 (+**0.9**)	58.7 (+**0.5**)	42.2 (+**1.0**)	34.1 (+**0.8**)	55.4 (+**0.7**)	36.4 (+**1.2**)

（c）Mask R-CNN，R50-C4，**1**× 训练日程

预训练	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
随机初始化	35.6	54.6	38.2	31.4	51.5	33.5
有监督 IN-1M	40.0	59.9	43.1	34.7	56.5	36.9
MoCo IN-1M	40.7 (+**0.7**)	60.5 (+**0.6**)	44.1 (+**1.0**)	35.4 (+**0.7**)	57.3 (+**0.8**)	37.6 (+**0.7**)
MoCo IG-1B	41.1 (+**1.1**)	60.7 (+**0.8**)	44.8 (+**1.7**)	35.6 (+**0.9**)	57.4 (+**0.9**)	38.1 (+**1.2**)

（d）Mask R-CNN，R50-C4，**2**× 训练日程

表5：在 COCO 上微调的目标检测与实例分割结果：在 val2017 上评估边界框 AP（AP^{bb}）和掩码 AP（AP^{mk}）。括号中为相对于对应 ImageNet 有监督预训练模型的差值；至少达到 +**0**.5个百分点的增益用绿色表示。

预训练	**COCO 关键点检测** AP^{kp}	AP^{kp}_{50}	AP^{kp}_{75}
随机初始化	65.9	86.5	71.7
有监督 IN-1M	65.8	86.9	71.9
MoCo IN-1M	66.8 (+**1.0**)	87.4 (+**0.5**)	72.5 (+**0.6**)
MoCo IG-1B	66.9 (+**1.1**)	87.8 (+**0.9**)	73.0 (+**1.1**)

预训练	**COCO 密集姿态估计** AP^{dp}	AP^{dp}_{50}	AP^{dp}_{75}
随机初始化	39.4	78.5	35.1
有监督 IN-1M	48.3	85.6	50.6
MoCo IN-1M	50.1 (+**1.8**)	86.8 (+**1.2**)	53.9 (+**3.3**)
MoCo IG-1B	50.6 (+**2.3**)	87.0 (+**1.4**)	54.3 (+**3.7**)

预训练	**LVIS v0.5 实例分割** AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
随机初始化	22.5	34.8	23.8
有监督 IN-1M†	24.4	37.8	25.8
MoCo IN-1M	24.1 (−0.3)	37.4 (−0.4)	25.5 (−0.3)
MoCo IG-1B	24.9 (+**0.5**)	38.2 (+0.4)	26.4 (+**0.6**)

预训练	**Cityscapes 实例分割** AP^{mk}	AP^{mk}_{50}	**语义分割（mIoU）** **Cityscapes**	**VOC**
随机初始化	25.4	51.1	65.3	39.5
有监督 IN-1M	32.9	59.6	74.6	74.4
MoCo IN-1M	32.3 (−0.6)	59.3 (−0.3)	75.3 (+**0.7**)	72.5 (−1.9)
MoCo IG-1B	32.9 (0.0)	60.3 (+**0.7**)	75.5 (+**0.9**)	73.6 (−0.8)

表6：MoCo 与 ImageNet 有监督预训练在多种任务上的微调比较。每个任务的所有条目使用相同架构和训练日程（见附录）。括号中为相对于对应 ImageNet 有监督预训练模型的差值；至少达到 +**0**.5个百分点的增益用绿色表示。

†：此条目冻结 BN，能改善结果；见正文。

使用 ImageNet 有监督预训练：

COCO 关键点检测：有监督预训练相对随机初始化没有明显优势，而 MoCo 在所有指标上均更好。

COCO 密集姿态估计 [1]：在这一对定位高度敏感的任务中，MoCo 明显优于有监督预训练，例如 AP^{dp}_{75} 提高3.7个百分点。

LVIS v0.5 实例分割 [27]：该任务包含~1000 个呈长尾分布的类别。针对 LVIS 的 ImageNet 有监督基线，我们发现冻结 BN 进行微调（24.4 AP^{mk}）优于可训练的 BN（细节见附录）。因此，我们在该任务上将 MoCo 与表现更好的有监督预训练变体比较。IG-1B 预训练的 MoCo 在所有指标上均超过它。

Cityscapes 实例分割 [10]：IG-1B 预训练的 MoCo 在 AP^{mk} 上与对应有监督预训练模型相当，在 AP^{mk}_{50} 上则更高。

语义分割：在 Cityscapes [10], 上，MoCo 最多比对应有监督预训练模型高0.9个百分点。但在 VOC 语义分割上，MoCo 至少低0.8个百分点，这是我们观察到的一个负面案例。

小结。MoCo 在7项检测或分割任务上可超过对应的 ImageNet 有监督预训练模型。[5] 此外，MoCo 在 Cityscapes 实例分割上与之相当，在 VOC 语义分割上则落后；附录中还给出了在 iNaturalist [57] 上表现相当的例子。总体而言，MoCo 在多项视觉任务上大幅缩小了无监督与有监督表征学习的差距。

值得关注的是，在所有这些任务中，IG-1B 预训练的 MoCo 始终优于 IN-1M 预训练的 MoCo。这说明 MoCo 能在这种大规模、较少人工整理的数据集上取得良好表现，体现了面向真实世界无监督学习的场景。

5 讨论与结论

本方法在多种视觉任务与数据集上取得积极的无监督学习结果，但仍存在开放问题。从 IN-1M 到 IG-1B 的改善稳定可见，却相对较小，说明更大数据可能尚未被充分利用。我们期待更先进的代理任务改善这一点。除了简单的实例判别 [61], , MoCo 也可能用于掩码自编码等代理任务，如语言 [12] 与视觉 [46] 中的做法。希望 MoCo 也能帮助其他涉及对比学习的代理任务。

[5]即 VOC／COCO 目标检测、COCO／LVIS 实例分割、COCO 关键点检测、COCO 密集姿态估计，以及 Cityscapes 语义分割。

A. Appendix

A.1. Implementation: Object detection backbones

The R50-dilated-C5 and R50-C4 backbones are similar to those available in `Detectron2` [60]: (i) *R50-dilated-C5*: the backbone includes the ResNet conv_5 stage with a dilation of 2 and stride 1, followed by a 3×3 convolution (with BN) that reduces dimension to 512. The box prediction head consists of two hidden fully-connected layers. (ii) *R50-C4*: the backbone ends with the conv_4 stage, and the box prediction head consists of the conv_5 stage (including global pooling) followed by a BN layer.

A.2. Implementation: COCO keypoint detection

We use Mask R-CNN (keypoint version) with R50-FPN, implemented in [60], fine-tuned on COCO `train2017` and evaluated on `val2017`. The schedule is 2×.

A.3. Implementation: COCO dense pose estimation

We use DensePose R-CNN [1] with R50-FPN, implemented in [60], fine-tuned on COCO `train2017` and evaluated on `val2017`. The schedule is "s1×".

A.4. Implementation: LVIS instance segmentation

We use Mask R-CNN with R50-FPN, fine-tuned in LVIS [27] `train_v0.5` and evaluated in `val_v0.5`. We follow the baseline in [27] (arXiv v3 Appendix B).

LVIS is a new dataset and model designs on it are to be explored. The following table includes the relevant ablations (all are averages of 5 trials):

pre-train	BN	1× schedule			2× schedule		
		AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
super. IN-1M	frozen	24.1	37.3	25.4	24.4	37.8	25.8
super. IN-1M	tuned	23.5	36.6	24.8	23.2	36.0	24.4
MoCo IN-1M	tuned	23.2	36.0	24.7	24.1	37.4	25.5
MoCo IG-1B	tuned	24.3	37.4	25.9	**24.9**	**38.2**	**26.4**

A supervised pre-training baseline, end-to-end tuned but with BN frozen, has 24.4 AP^{mk}. But tuning BN in this baseline leads to worse results and overfitting (this is unlike on COCO/VOC where tuning BN gives better or comparable accuracy). MoCo has 24.1 AP^{mk} with IN-1M and 24.9 AP^{mk} with IG-1B, both outperforming the supervised pre-training counterpart under the same tunable BN setting. Under the best individual settings, MoCo can still outperform the supervised pre-training case (24.9 *vs*. 24.4, as reported in Table 6 in Sec 4.2).

A.5. Implementation: Semantic segmentation

We use an FCN-based [43] structure. The backbone consists of the convolutional layers in R50, and the 3×3 convolutions in conv_5 blocks have dilation 2 and stride 1. This is followed by two extra 3×3 convolutions of 256 channels, with BN and ReLU, and then a 1×1 convolution for per-pixel classification. The total stride is 16 (FCN-16s [43]). We set dilation = 6 in the two extra 3×3 convolutions, following the large field-of-view design in [6].

Training is with random scaling (by a ratio in [0.5, 2.0]), cropping, and horizontal flipping. The crop size is 513 on VOC and 769 on Cityscapes [6]. Inference is performed on the original image size. We train with mini-batch size 16 and weight decay 0.0001. Learning rate is 0.003 on VOC and is 0.01 on Cityscapes (multiplied by 0.1 at 70-th and 90-th percentile of training). For VOC, we train on the `train_aug2012` set (augmented by [30], 10582 images) for 30k iterations, and evaluate on `val2012`. For Cityscapes, we train on the `train_fine` set (2975 images) for 90k iterations, and evaluate on the `val` set. Results are reported as averages over 5 trials.

A.6. iNaturalist fine-grained classification

In addition to the detection/segmentation experiments in the main paper, we study fine-grained classification on the iNaturalist 2018 dataset [57]. We fine-tune the pre-trained models end-to-end on the `train` set (~437k images, 8142 classes) and evaluate on the `val` set. Training follows the typical ResNet implementation in PyTorch with 100 epochs. Fine-tuning has a learning rate of 0.025 (*vs*. 0.1 from scratch) decreased by 10 at the 70-th and 90-th percentile of training. The following is the R50 result:

pre-train	rand init.	$\text{super.}_{\text{IN-1M}}$	$\textbf{MoCo}_{\text{IN-1M}}$	$\textbf{MoCo}_{\text{IG-1B}}$
accuracy (%)	61.8	**66.1**	65.6	65.8

MoCo is ~4% better than training from random initialization, and is closely comparable with its ImageNet supervised counterpart. This again shows that MoCo unsupervised pre-training is competitive.

A.7. Fine-tuning in ImageNet

Linear classification on frozen features (Sec. 4.1) is a common protocol of evaluating unsupervised pre-training methods. However, in practice, it is more common to fine-tune the features end-to-end in a downstream task. For completeness, the following table reports end-to-end fine-tuning results for the 1000-class ImageNet classification, compared with training from scratch (fine-tuning uses an initial learning rate of 0.03, *vs*. 0.1 from scratch):

pre-train	random init.	$\textbf{MoCo}_{\text{IG-1B}}$
accuracy (%)	76.5	**77.3**

As here ImageNet is the downstream task, the case of MoCo pre-trained on IN-1M does not represent a real scenario (for reference, we report that its accuracy is 77.0% after fine-tuning). But unsupervised pre-training in the *separate*, unlabeled dataset of IG-1B represents a typical scenario: in this case, MoCo improves by 0.8%.

A 附录

A.1 实现：目标检测骨干网络

R50-dilated-C5 与 R50-C4 类似 Detectron 2 [60]提供的主干。（i）R50-dilated-C5 包括膨胀率2、步幅1的 ResNet conv_5 阶段，再接一个带 BN 的 3×3 卷积，将维度降为512。框预测头含两个隐藏全连接层。（ii）R50-C4 主干止于 conv_4 阶段，框预测头为 conv_5 阶段，包括全局池化，再接 BN。

A.2 实现：COCO 关键点检测

我们使用 R50-FPN 骨干的 Mask R-CNN（关键点版本），采用 [60], 中的实现，在 COCO train2017 上微调，在 val2017 上评估。训练日程为 2×。

A.3 实现：COCO 密集姿态估计

我们使用 R50-FPN 骨干的 DensePose R-CNN [1]，采用 [60], 中的实现，在 COCO train2017 上微调，在 val2017 上评估。训练日程为“s1×”。

A.4 实现：LVIS 实例分割

我们使用 R50-FPN 骨干的 Mask R-CNN，在 LVIS[27] train v0.5 上微调，在 val v0.5 上评估，遵循 [27] 中的基线（arXiv 第3版附录B）。

LVIS 是新数据集，其模型设计仍待探索。下表包含相关消融结果（均为5次试验平均值）：

预训练	BN	1× 训练日程			2× 训练日程		
		AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
有监督 IN-1M	冻结	24.1	37.3	25.4	24.4	37.8	25.8
有监督 IN-1M	微调	23.5	36.6	24.8	23.2	36.0	24.4
MoCo IN-1M	微调	23.2	36.0	24.7	24.1	37.4	25.5
MoCo IG-1B	微调	24.3	37.4	25.9	**24.9**	**38.2**	**26.4**

监督预训练基线端到端微调但冻结 BN，达到24.4 AP^{mk}；允许 BN 更新则导致更差结果及过拟合，这不同于 COCO/VOC，后者更新 BN 能取得更好或相近准确率。MoCo 使用 IN-1M 达到24.1 AP^{mk}，使用 IG-1B 达到24.9 AP^{mk}，在相同可更新 BN 设置中都优于监督对应模型。各自采用最佳设置时，MoCo 仍优于监督预训练，24.9对24.4，见第4.2节表6。

A.5 实现：语义分割

我们采用基于 FCN [43] 的结构。骨干包含 R50 的卷积层，conv_5 模块中的 3×3 卷积采用膨胀率2、步幅1。之后接两个额外的256通道 3×3 卷积，及 BN 和 ReLU，然后用一个 1×1 卷积进行逐像素分类。总步幅为16（FCN-16s [43]）。按照 [6]的大视野设计，两个额外的 3×3 卷积采用膨胀率 = 6 。

训练使用比例范围为 [0.5，2.0] 的随机缩放、裁剪及水平翻转。VOC 裁剪513，Cityscapes 裁剪769 [6]；推理使用原图尺寸。批量16，权重衰减0.0001；VOC 学习率0.003，Cityscapes 为0.01，在训练进度70%及90%时乘0.1。VOC 使用由额外图像 [30], 10582 扩充的 train_aug2012，训练30000次，在 val2012 评估。Cityscapes 使用2975张 train_fine 图像，训练90000次，在 val 上评估。报告5次运行平均值。

A.6 iNaturalist 细粒度分类

除正文检测分割实验外，我们还在 iNaturalist 2018 [57]上研究细粒度分类。在包含 ~437 千张图像、8142类的训练集上端到端微调，在 val 上评估。训练遵循 PyTorch 常见 ResNet 实现，共100轮。微调学习率0.025，从头训练则为0.1；在70%和90%进度时除以10。R50 结果如下：

预训练	随机初始化	有监督$_{\text{IN-1M}}$	$\textbf{MoCo}_{\text{IN-1M}}$	$\textbf{MoCo}_{\text{IG-1B}}$
准确率（%）	61.8	**66.1**	65.6	65.8

MoCo 比随机初始化训练高 ~4个百分点，与对应 ImageNet 有监督预训练模型的表现十分接近。这再次表明，MoCo 无监督预训练具有竞争力。

A.7 在 ImageNet 上微调

在冻结特征上进行线性分类（第4.1节）是评估无监督预训练方法的常用协议。但在实践中，更常见的做法是在下游任务中端到端微调特征。为完整起见，下表报告了 ImageNet 1000类分类的端到端微调结果，并与从零训练比较（微调初始学习率为0.03，从零训练为0.1）：

预训练	随机初始化	$\textbf{MoCo}_{\text{IG-1B}}$
准确率（%）	76.5	**77.3**

由于此处 ImageNet 是下游任务，在 IN-1M 上预训练 MoCo 的情形并不代表真实的迁移场景（供参考，其微调后准确率为77.0%）。而在独立、无标签的 IG-1B 数据集上无监督预训练则代表典型场景；此时 MoCo 提高了0.8个百分点。

pre-train	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
random init.	36.7	56.7	40.0	33.7	53.8	35.9
super. IN-1M	40.6	61.3	44.4	36.8	58.1	39.5
MoCo IN-1M	40.8 (+0.2)	61.6 (+0.3)	44.7 (+0.3)	36.9 (+0.1)	58.4 (+0.3)	39.7 (+0.2)
MoCo IG-1B	41.1 (+0.5)	61.8 (+0.5)	45.1 (+0.7)	37.4 (+0.6)	59.1 (+1.0)	40.2 (+0.7)

(a) Mask R-CNN, R50-FPN, **2**× schedule

AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
41.4	61.9	45.1	37.6	59.1	40.3
41.9	62.5	45.6	38.0	59.6	40.8
42.3 (+0.4)	62.7 (+0.2)	46.2 (+0.6)	38.3 (+0.3)	60.1 (+0.5)	41.2 (+0.4)
42.8 (+0.9)	63.2 (+0.7)	47.0 (+1.4)	38.7 (+0.7)	60.5 (+0.9)	41.3 (+0.5)

(b) Mask R-CNN, R50-FPN, **6**× schedule

Table A.1. Object detection and instance segmentation fine-tuned on COCO: **2**× ***vs.*** **6**× **schedule**. In the brackets are the gaps to the ImageNet supervised pre-training counterpart. In green are the gaps of at least **+0.5** point.

A.8. COCO longer fine-tuning

In Table 5 we reported results of the 1× (~12 epochs) and 2× schedules on COCO. These schedules were inherited from the original Mask R-CNN paper [32], which could be suboptimal given later advance in the field. In Table A.1, we supplement the results of a 6× schedule (~72 epochs) [31] and compare with those of the 2× schedule.

We observe: (i) fine-tuning with ImageNet-supervised pre-training still has improvements (41.9 AP^{bb}); (ii) training from scratch largely catches up (41.4 AP^{bb}); (iii) the MoCo counterparts improve further (*e.g.*, to 42.8 AP^{bb}) and have larger gaps (*e.g.*, +0.9 AP^{bb} with 6×, *vs.* +0.5 AP^{bb} with 2×). Table A.1 and Table 5 suggest that the MoCo pre-trained features can have *larger* advantages than the ImageNet-supervised features when fine-tuning *longer*.

A.9. Ablation on Shuffling BN

Figure A.1 provides the training curves of MoCo with or without shuffling BN: removing shuffling BN shows obvious overfitting to the pretext task: training accuracy of the pretext task (dash curve) quickly increases to >99.9%, and the kNN-based validation classification accuracy (solid curve) drops soon. This is observed for both the MoCo and end-to-end variants; the memory bank variant implicitly has different statistics for q and k, so avoids this issue.

These experiments suggest that without shuffling BN, the sub-batch statistics can serve as a "signature" to tell which sub-batch the positive key is in. Shuffling BN can remove this signature and avoid such cheating.

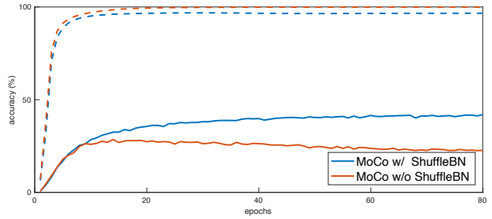


Figure A.1. **Ablation of Shuffling BN**. *Dash*: training curve of the pretext task, plotted as the accuracy of (K+1)-way dictionary lookup. *Solid*: validation curve of a kNN-based monitor [61] (not a linear classifier) on ImageNet classification accuracy. This plot shows the first 80 epochs of training: training longer without shuffling BN overfits more.

References

[1] Rıza Alp Güler, Natalia Neverova, and Iasonas Kokkinos. DensePose: Dense human pose estimation in the wild. In *CVPR*, 2018.

[2] Philip Bachman, R Devon Hjelm, and William Buchwalter. Learning representations by maximizing mutual information across views. *arXiv:1906.00910*, 2019.

[3] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. Deep clustering for unsupervised learning of visual features. In *ECCV*, 2018.

[4] Mathilde Caron, Piotr Bojanowski, Julien Mairal, and Armand Joulin. Unsupervised pre-training of image features on non-curated data. In *ICCV*, 2019.

[5] Ken Chatfield, Victor Lempitsky, Andrea Vedaldi, and Andrew Zisserman. The devil is in the details: an evaluation of recent feature encoding methods. In *BMVC*, 2011.

[6] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L Yuille. DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs. *TPAMI*, 2017.

[7] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. *arXiv:2002.05709*, 2020.

[8] Xinlei Chen, Haoqi Fan, Ross Girshick, and Kaiming He. Improved baselines with momentum contrastive learning. *arXiv:2003.04297*, 2020.

[9] Adam Coates and Andrew Ng. The importance of encoding versus training with sparse coding and vector quantization. In *ICML*, 2011.

[10] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The Cityscapes dataset for semantic urban scene understanding. In *CVPR*, 2016.

[11] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.

[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.

[13] Carl Doersch, Abhinav Gupta, and Alexei A Efros. Unsupervised visual representation learning by context prediction. In *ICCV*, 2015.

[14] Carl Doersch and Andrew Zisserman. Multi-task self-supervised visual learning. In *ICCV*, 2017.

[15] Jeff Donahue, Philipp Krähenbühl, and Trevor Darrell. Adversarial feature learning. In *ICLR*, 2017.

预训练	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
随机初始化	36.7	56.7	40.0	33.7	53.8	35.9
有监督 IN-1M	40.6	61.3	44.4	36.8	58.1	39.5
MoCo IN-1M	40.8 (+0.2)	61.6 (+0.3)	44.7 (+0.3)	36.9 (+0.1)	58.4 (+0.3)	39.7 (+0.2)
MoCo IG-1B	41.1 (+0.5)	61.8 (+0.5)	45.1 (+0.7)	37.4 (+0.6)	59.1 (+1.0)	40.2 (+0.7)

（a）Mask R-CNN，R50-FPN，**2**× 训练日程

AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}	AP^{mk}	AP^{mk}_{50}	AP^{mk}_{75}
41.4	61.9	45.1	37.6	59.1	40.3
41.9	62.5	45.6	38.0	59.6	40.8
42.3 (+0.4)	62.7 (+0.2)	46.2 (+0.6)	38.3 (+0.3)	60.1 (+0.5)	41.2 (+0.4)
42.8 (+0.9)	63.2 (+0.7)	47.0 (+1.4)	38.7 (+0.7)	60.5 (+0.9)	41.3 (+0.5)

（b）Mask R-CNN，R50-FPN，**6**× 训练日程

表 A.1：在 COCO 上微调的目标检测与实例分割结果：比较 **2**× 与 **6**× 训练日程。括号中为相对于对应 ImageNet 有监督预训练模型的差值。至少达到 **+0**.5个百分点的增益用绿色表示。

A.8 COCO 上更长的微调日程

表5报告了 COCO 上 1× （~12轮）和 2× 训练日程的结果。这些日程沿用了最初的 Mask R-CNN 论文 [32], ; 考虑到此后领域的发展，它们可能并非最优。表 A.1 补充了 6× 训练日程（~72轮）[31] 的结果，并与 2×训练日程比较。

我们观察到：（i）ImageNet 监督预训练的微调仍有改善，达到41.9 AP^{bb}；（ii）从头训练大幅追近，达到 41.4 AP^{bb}；（iii）MoCo 进一步改善，例如42.8 AP^{bb}，差距更大，例如 6× 下提升 +0.9 AP^{bb}，而 2× 下提升 +0.5 AP^{bb}。表A.1与表5说明，更长微调时，MoCo 特征相对 ImageNet 监督特征可能具有更大优势。

A.9 BN 打乱的消融

图 A.1 展示了使用和不使用 BN 打乱的 MoCo 训练曲线。去除 BN 打乱后，代理任务出现明显过拟合：代理任务的训练准确率（虚线）迅速升至 >99.9%，而基于 kNN 的验证分类准确率（实线）很快下降。MoCo 和端到端变体中都能观察到这一现象；记忆库变体隐含地为 q 和 k使用不同统计量，因此避免了这一问题。

这些实验表明，不打乱 BN 时，子批次统计量可以成为识别正键所属子批次的"指纹"。打乱 BN 能消除这一指纹，避免此类作弊。

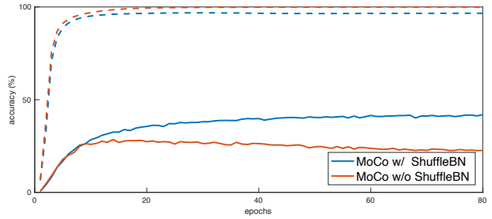


图 A.1：BN 打乱的消融。虚线为代理任务训练曲线，以（K+1）分类的字典查询准确率表示。实线为基于 kNN 的监测器 [61]（并非线性分类器）在 ImageNet 上的验证分类准确率。图中展示前80轮训练：不打乱 BN 时，训练越久，过拟合越严重。

参考文献

[1] Rıza Alp G¨uler, Natalia Neverova, and Iasonas Kokkinos. DensePose: Dense human pose estimation in the wild. In CVPR, 2018.[2] Philip Bachman, R Devon Hjelm, and William Buchwalter. Learning representations by maximizing mutual information across views. arXiv:1906.00910, 2019.[3] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. Deep clustering for unsupervised learning of visual features. In ECCV, 2018.[4] Mathilde Caron, Piotr Bojanowski, Julien Mairal, and Ar- mand Joulin. Unsupervised pre-training of image features on non-curated data. In ICCV, 2019.[5] Ken Chatfield, Victor Lempitsky, Andrea Vedaldi, and An- dre w Zisserman. The devil is in the details: an evaluation of recent feature encoding methods. In BMVC, 2011.[6] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L Yuille. DeepLab: Semantic image segmentation with deep convolutional nets, atrous con- volution, and fully connected CRFs. TPAMI, 2017.[7] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive lear ning of visual representations. arXiv:2002.05709, 2020.[8] Xinlei Chen, Haoqi Fan, Ross Girshick, and Kaiming He. Improved baselines with momentum contrastive learning. arXiv:2003.04297, 2020.[9] Adam Coates and Andrew Ng. The importance of encoding versus training with sparse coding and vector quantization. In ICML, 2011.[10] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The Cityscapes dataset for semantic urban scene understanding. In CVPR, 2016.[11] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In CVPR, 2009.[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional trans- formers for language understanding. In NAACL, 2019.[13] Carl Doersch, Abhinav Gupta, and Alexei A Efros. Unsupervised visual representation learning by context prediction. In ICCV, 2015.[14] Carl Doersch and Andrew Zisserman. Multi-task self- supervised visual learning. In ICCV, 2017.[15] Jeff Donahue, Philipp Kr¨ahenb¨uhl, and Trevor Darrell. Ad- versarial feature learning. In ICLR, 2017.

[16] Jeff Donahue and Karen Simonyan. Large scale adversarial representation learning. *arXiv:1907.02544*, 2019.

[17] Alexey Dosovitskiy, Jost Tobias Springenberg, Martin Riedmiller, and Thomas Brox. Discriminative unsupervised feature learning with convolutional neural networks. In *NeurIPS*, 2014.

[18] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The Pascal Visual Object Classes (VOC) Challenge. *IJCV*, 2010.

[19] Spyros Gidaris, Praveer Singh, and Nikos Komodakis. Unsupervised representation learning by predicting image rotations. In *ICLR*, 2018.

[20] Ross Girshick. Fast R-CNN. In *ICCV*, 2015.

[21] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.

[22] Ross Girshick, Ilija Radosavovic, Georgia Gkioxari, Piotr Dollár, and Kaiming He. Detectron, 2018.

[23] Aidan N Gomez, Mengye Ren, Raquel Urtasun, and Roger B Grosse. The reversible residual network: Backpropagation without storing activations. In *NeurIPS*, 2017.

[24] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *NeurIPS*, 2014.

[25] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv:1706.02677*, 2017.

[26] Priya Goyal, Dhruv Mahajan, Abhinav Gupta, and Ishan Misra. Scaling and benchmarking self-supervised visual representation learning. In *ICCV*, 2019.

[27] Agrim Gupta, Piotr Dollar, and Ross Girshick. LVIS: A dataset for large vocabulary instance segmentation. In *CVPR*, 2019.

[28] Michael Gutmann and Aapo Hyvärinen. Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. In *AISTATS*, 2010.

[29] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *CVPR*, 2006.

[30] Bharath Hariharan, Pablo Arbeláez, Lubomir Bourdev, Subhransu Maji, and Jitendra Malik. Semantic contours from inverse detectors. In *ICCV*, 2011.

[31] Kaiming He, Ross Girshick, and Piotr Dollár. Rethinking ImageNet pre-training. In *ICCV*, 2019.

[32] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask R-CNN. In *ICCV*, 2017.

[33] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[34] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *ECCV*, 2016.

[35] Olivier J Hénaff, Ali Razavi, Carl Doersch, SM Eslami, and Aaron van den Oord. Data-efficient image recognition with contrastive predictive coding. *arXiv:1905.09272*, 2019. Updated version accessed at `https://openreview.net/pdf?id=rJerHlrYwH`.

[36] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Adam Trischler, and Yoshua Bengio. Learning deep representations by mutual information estimation and maximization. In *ICLR*, 2019.

[37] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[38] Alexander Kolesnikov, Xiaohua Zhai, and Lucas Beyer. Revisiting self-supervised visual representation learning. In *CVPR*, 2019.

[39] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1989.

[40] Sungbin Lim, Ildoo Kim, Taesup Kim, Chiheon Kim, and Sungwoong Kim. Fast AutoAugment. *arXiv:1905.00397*, 2019.

[41] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *CVPR*, 2017.

[42] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft COCO: Common objects in context. In *ECCV*, 2014.

[43] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.

[44] Dhruv Mahajan, Ross Girshick, Vignesh Ramanathan, Kaiming He, Manohar Paluri, Yixuan Li, Ashwin Bharambe, and Laurens van der Maaten. Exploring the limits of weakly supervised pretraining. In *ECCV*, 2018.

[45] Mehdi Noroozi and Paolo Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. In *ECCV*, 2016.

[46] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv:1807.03748*, 2018.

[47] Deepak Pathak, Ross Girshick, Piotr Dollár, Trevor Darrell, and Bharath Hariharan. Learning features by watching objects move. In *CVPR*, 2017.

[48] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A Efros. Context encoders: Feature learning by inpainting. In *CVPR*, 2016.

[49] Chao Peng, Tete Xiao, Zeming Li, Yuning Jiang, Xiangyu Zhang, Kai Jia, Gang Yu, and Jian Sun. MegDet: A large mini-batch object detector. In *CVPR*, 2018.

[50] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018.

[51] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[52] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015.

[16] Jeff Donahue and Karen Simonyan. Large scale adversarial representation learning. arXiv:1907.02544, 2019.[17] Alexey Dosovitskiy, Jost Tobias Springenberg, Martin Ried- miller, and Thomas Brox. Discriminative unsupervised feature learning with convolutional neural networks. In NeurIPS, 2014.[18] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The Pascal Visual Ob- ject Classes (VOC) Challenge. IJCV, 2010.[19] Spyros Gidaris, Praveer Singh, and Nikos Komodakis. Un- supervised representation learning by predicting image rota- tions. In ICLR, 2018.[20] Ross Girshick. Fast R-CNN. In ICCV, 2015.[21] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate o bject detection and semantic segmentation. In CVPR, 2014.[22] Ross Girshick, Ilija Radosavovic, Georgia Gkioxari, Piotr Doll´ar, and Kaiming He. Detectron, 2018.[23] Aidan N Gomez, Mengye Ren, Raquel Urtasun, and Roger B Grosse. The reversible residual network: Backpropagation without storing activations. In NeurIPS, 2017.[24] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In NeurIPS, 2014.[25] Priya Goyal, Piotr Doll´ar, Ross Girshick, Pieter Noord- huis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. arXiv:1706.02677, 2017.[26] Priya Goyal, Dhruv Mahajan, Abhinav Gupta, and Ishan Misra. Scaling and benchmarking self-supervised visual rep- resentation learning. In ICCV, 2019.[27] Agrim Gupta, Piotr Dollar, and Ross Girshick. LVIS: A dataset for large vocabulary i nstance segmentation. In CVPR, 2019.[28] Michael Gutmann and Aapo Hyv¨arinen. Noise-contrastive estimation: A new estimation principle for unnormalized sta- tistical models. In AISTATS, 2010.[29] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimension- ality reduction by learning an invariant mapping. In CVPR, 2006.[30]Bharath Hariharan, Pablo Arbel´aez, Lubomir Bourdev, Subhransu Maji, and Jitendra Malik. Semantic contours from inverse detectors. In ICCV, 2011.[31] Kaiming He, Ross Girshick, and Piotr Doll´ar. Rethinking ImageNet pre-training. In ICCV, 2019.[32] Kaiming He, Georgia Gkioxari, Piotr Doll´ar, and Ross Gir- shick. Mask R-CNN. In ICCV, 2017.[33] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In CVPR, 2016.[34] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In ECCV, 2016.[35] Olivier J H´enaff, Ali Razavi, Carl Doersch, SM Eslami, and Aaron van den Oord. Data-efficient image recognition with contrastive predictive coding. arXiv:1905.09272, 2019. Up- dated version accessed at https://openreview.net/pdf?id=rJerHlrYwH.[36] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Adam Trischler, and Yoshua Bengio. Learn- ing deep representations by mutual information estimation and maximization. In ICLR, 2019.[37] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal co- variate shift. In ICML, 2015.[38] Alexander Kolesnikov, Xiaohua Zhai, and Lucas Beyer. Re- visiting self-supervised visual representation learning. In CVPR, 2019.[39]Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwrit- ten zip code recognition. Neural computation, 1989.[40] Sungbin Lim, Ildoo Kim, Taesup Kim, Chiheon Kim, and Sungwoong Kim. Fast AutoAugment. arXiv:1905 00397, 2019 . .[41] Tsung-Yi Lin, Piotr Doll´ar, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In CVPR, 2017.[42] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Doll´ar, and C Lawrence Zitnick. Microsoft COCO: Common objects in context. In ECCV, 2014.[43] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In CVPR, 2015.[44]Dhruv Mahajan, Ross Girshick, Vignesh Ramanathan, Kaiming He, Manohar Paluri, Yixuan Li, Ashwin Bharambe, and Laurens van der Maaten. Exploring the limits of weakly supervised pretraining. In ECCV, 2018. [45] Mehdi Noroozi and Paolo Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. In ECCV, 2016.[46] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Rep- resentation learning with contrastive predictive coding. arXiv:1807.03748, 2018.[47] Deepak Pathak, Ross Girshick, Piotr Doll´ar, Trevor Darrell, and Bharath Hariharan. Learning features by watching ob- jects move. In CVPR, 2017.[48] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A Efros. Context encoders: Feature learning by inpainting. In CVPR, 2016.[49] Chao Peng, Tete Xiao, Zeming Li, Yuning Jiang, Xiangyu Zhang, Kai Jia, Gang Yu, and Jian Sun. MegDet: A large mini-batch object detector. In CVPR, 2018.[50] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018.[51] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsuper- vised multitask learners. 2019.[52] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with re- gion proposal networks. In NeurIPS, 2015.

[53] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[54] Josef Sivic and Andrew Zisserman. Video Google: a text retrieval approach to object matching in videos. In *ICCV*, 2003.

[55] Bart Thomee, David A Shamma, Gerald Friedland, Benjamin Elizalde, Karl Ni, Douglas Poland, Damian Borth, and Li-Jia Li. YFCC100M: The new data in multimedia research. *Communications of the ACM*, 2016.

[56] Yonglong Tian, Dilip Krishnan, and Phillip Isola. Contrastive multiview coding. *arXiv:1906.05849*, 2019. Updated version accessed at https://openreview.net/pdf?id=BkgStySKPB.

[57] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The iNaturalist species classification and detection dataset. In *CVPR*, 2018.

[58] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *ICML*, 2008.

[59] Xiaolong Wang and Abhinav Gupta. Unsupervised learning of visual representations using videos. In *ICCV*, 2015.

[60] Yuxin Wu, Alexander Kirillov, Francisco Massa, Wan-Yen Lo, and Ross Girshick. Detectron2. https://github.com/facebookresearch/detectron2, 2019.

[61] Zhirong Wu, Yuanjun Xiong, Stella Yu, and Dahua Lin. Unsupervised feature learning via non-parametric instance discrimination. In *CVPR*, 2018. Updated version accessed at: https://arxiv.org/abs/1805.01978v1.

[62] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *CVPR*, 2017.

[63] Mang Ye, Xu Zhang, Pong C Yuen, and Shih-Fu Chang. Unsupervised embedding learning via invariant and spreading instance feature. In *CVPR*, 2019.

[64] Richard Zhang, Phillip Isola, and Alexei A Efros. Colorful image colorization. In *ECCV*, 2016.

[65] Richard Zhang, Phillip Isola, and Alexei A Efros. Split-brain autoencoders: Unsupervised learning by cross-channel prediction. In *CVPR*, 2017.

[66] Chengxu Zhuang, Alex Lin Zhai, and Daniel Yamins. Local aggregation for unsupervised learning of visual embeddings. In *ICCV*, 2019. Additional results accessed from supplementary materials.

[53] Karen Simonyan and Andrew Zisserman. Very deep convo- lutional networks for large-scale image recognition. In ICLR, 2015.[54] Josef Sivic and Andrew Zisserman. Video Google: a text retrieval approach to object matching in videos. In ICCV, 2003.[55] Bart Thomee, David A Shamma, Gerald Friedland, Ben- jamin Elizalde, Karl Ni, Douglas Poland, Damian Borth, and Li-Jia Li. YFCC100M: The new data in multimedia research. Communications of the AC M, 2016.[56] Yonglong Tian, Dilip Krishnan, and Phillip Isola. Con- trastive multiview coding. arXiv:1906.05849, 2019. Updated version accessed at https://openreview.net/pdf? id=BkgStySKPB.[57] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The iNaturalist species classification and detection dataset. In CVPR, 2018.[58] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In ICML, 2008.[59] Xiaolong Wang and Abhinav Gupta. Unsupervised learning of visual representations using videos. In ICCV, 2015.[60] Yuxin Wu, Alexander Kirillov, Francisco Massa, Wan-Yen Lo, and Ross Girshick. Detectron2. https://github. com/facebookresearch/detectron2, 2019.[61] Zhirong Wu, Yuanjun Xiong, Stella Yu, and Dahua Lin. Un- supervised feature learning via non-parametric instance dis- crimination. In CVPR, 2018. Updated version accessed at: https://arxiv.org/abs/1805.01978v1.[62] Saining Xie, Ross Girshick, Piotr Doll´ar, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In CVPR, 2017.[63] Mang Ye, Xu Zhang, Pong C Yuen, and Shih-Fu Chang. Un- supervised embedding learning via invariant and spreading instance feature. In CVPR, 2019.[64] Richard Zhang, Phillip Isola, and Alexei A Efros. Colorful image colorization. In ECCV, 2016.[65] Richard Zhang, Phillip Isola, and Alexei A Efros. Split-brain autoencoders: Unsupervised learning by cross-channel pre- diction. In CVPR, 2017.[66] Chengxu Zhuang, Alex Lin Zhai, and Daniel Yamins. Local aggregation for unsupervised learning of visual embeddings. In ICCV, 2019. Additional results accessed from supplemen- tary materials.